DAO COSMOS OS — THE ULTIMATE PROJECT PLAN

Complete Technical Blueprint, UI/UX Specification, Architecture Guide, and Hackathon War Manual

---

TABLE OF CONTENTS

1. Executive Summary & Hackathon Strategy
2. Complete Tech Stack Deep Dive
3. Architecture & System Design
4. Complete File Structure (Every Single File)
5. Database Schema & Data Models
6. UI/UX Design System — The Cosmos Design Language
7. Module 1: DAO Neural Network (Central Hub)
8. Module 2: Governance Arena
9. Module 3: Treasury Reactor
10. Module 4: Contributor Galaxy
11. Module 5: Security Sentinel
12. Module 6: AI Command Agents
13. Module 7: Multiverse DAO Simulator
14. Gamification System
15. 3D Engine & Visual Effects Bible
16. Blockchain Integration Layer
17. AI Integration Layer
18. Realtime Event System
19. API Routes & Backend Logic
20. Authentication & Wallet Connection
21. State Management Architecture
22. Performance Optimization
23. Deployment & DevOps
24. Division of Work (2 People — Hour by Hour)
25. Demo Script & Presentation Strategy
26. Risk Mitigation & Contingency Plans
27. Post-Hackathon Roadmap
28. Complete Component API Reference

1. EXECUTIVE SUMMARY & HACKATHON STRATEGY

1.1 What We Are Building

DAO Cosmos OS is a sci-fi operating system interface for decentralized autonomous organizations. It replaces traditional dashboard UIs with an immersive 3D space command center where every DAO operation is visualized through a cosmic metaphor system.

The core thesis: **DAOs are complex living systems. They deserve a living interface.**

1.2 Why This Wins Hackathons

Hackathon judges evaluate on 5 axes. Here is how we score on each:

Innovation (Score: 10/10): No one has built a 3D immersive DAO OS. Every competitor will ship a dashboard. We ship a universe. The metaphor mapping (treasury=reactor, governance=arena, contributors=galaxy) is original, internally consistent, and immediately understandable.

Technical Depth (Score: 9/10): We touch 3D rendering (Three.js/R3F), real-time systems (WebSocket), blockchain data (Alchemy), AI agents (Claude API), and simulation algorithms. This is the kind of multi-domain technical ambition that impresses judges who are themselves engineers.

Design & UX (Score: 10/10): The holographic design language, inspired by Iron Man's JARVIS, Mass Effect's Normandy, and Ender's Game's battle room, creates a visceral "wow" reaction. This is the single biggest differentiator. Most hackathon projects look ugly. Ours looks like a movie.

Completeness (Score: 7/10 — and that is okay): We will not finish everything. The strategy is to build 3 modules to perfection rather than 7 modules to 60%. Judges award polish, not breadth.

Demo Quality (Score: 10/10): The project is inherently cinematic. Every action — voting, transferring funds, detecting threats — has a visual spectacle attached to it. The demo basically runs itself.

1.3 The Golden Rule: Build for the Demo

Every single technical decision must pass one test: **"Does this make the 3-minute demo more impressive?"**

If YES → build it.
If NO → skip it, even if it is technically interesting.

1.4 Target DAO

We will visualize **Uniswap DAO** as our primary data source because it has rich governance data (hundreds of proposals with vote history), a massive treasury (~$3B+), active contributor ecosystem, publicly available on-chain data, and name recognition with judges.

Fallback: Aave DAO or ENS DAO if Uniswap data is harder to fetch.

1.5 Hackathon Timeline Overview

Assuming a 36-48 hour hackathon:

- Hours 0-12: Foundation (scaffold, 3D scene, data layer, design system)
- Hours 12-24: Core Modules (governance, treasury, AI agent)
- Hours 24-36: Secondary Modules + Polish (contributor galaxy, simulator, effects)
- Hours 36-48: Integration, Demo Prep, Deployment, Rehearsal

---

2. COMPLETE TECH STACK DEEP DIVE

2.1 Frontend Framework: Next.js 14 (App Router)

Why Next.js specifically:

- App Router gives us file-based routing (each module = one page route)
- Server Components for initial data fetching (blockchain data can be fetched server-side)
- API Routes for our backend (no separate backend server needed)
- Built-in image optimization
- Vercel deployment is zero-config
- TypeScript support out of the box

Configuration:

```typescript
```

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  transpilePackages: ['three'],
  webpack: (config) => {
    config.module.rules.push({
      test: /\.(glsl|vs|fs|vert|frag)$/,
      use: ['raw-loader', 'glslify-loader'],
    });
    return config;
  },
  images: {
    domains: ['raw.githubusercontent.com', 'avatars.githubusercontent.com'],
  },
};
module.exports = nextConfig;
```

2.2 3D Engine: React Three Fiber + Drei + Postprocessing

React Three Fiber (R3F) is a React renderer for Three.js. Instead of imperative Three.js code, we write declarative JSX. This is critical for hackathon speed — we can compose 3D scenes as React components.

Drei is a helper library for R3F that gives us:

- `<OrbitControls>` — camera controls for navigating the universe
- `<Stars>` — instant star field background
- `<Trail>` — energy trail effects behind moving objects
- `<Float>` — gentle floating animation for holographic panels
- `<Text>` — 3D text rendering
- `<Billboard>` — elements that always face the camera
- `<Line>` — for neural network connections
- `<Sphere>`, `<Box>` — primitive geometries
- `<Environment>` — lighting presets
- `<Html>` — embed HTML inside 3D scene (critical for our holographic panels)
- `<useGLTF>` — load 3D models

@react-three/postprocessing gives us:

- `<Bloom>` — glow effects (THE most important effect for our aesthetic)
- `<ChromaticAberration>` — RGB split effect for sci-fi feel
- `<Vignette>` — darkened edges

- `<Noise>` — film grain
- `<SSAO>` — ambient occlusion for depth
- `<GodRays>` — light beams from bright objects

Version pinning:

```json
{
  "@react-three/fiber": "^8.15.0",
  "@react-three/drei": "^9.92.0",
  "@react-three/postprocessing": "^2.16.0",
  "three": "^0.160.0"
}
```

2.3 2D Animation: Framer Motion

Used for all 2D overlay animations:

- Holographic panel entrance/exit (scale + opacity + blur)
- Card flip animations for proposals
- Sidebar slide transitions
- Number counting animations (treasury values)
- Staggered list animations
- Layout animations when switching modules

Key Framer Motion features we will use:

- `motion.div` — animated div
- `AnimatePresence` — exit animations
- `useMotionValue` + `useTransform` — reactive animations
- `variants` — complex multi-element orchestration
- `layout` — automatic layout transitions
- `whileHover`, `whileTap` — interaction states

2.4 State Management: Zustand

Zustand is chosen over Redux, Jotai, or React Context because:

- Zero boilerplate (one file per store)
- No providers needed
- Works outside React (we need this for Three.js callbacks)

- Built-in middleware (persist, devtools, immer)
- Subscriptions with selectors (performance)

We will have 4 stores:

1. `daoStore` — all DAO data (proposals, treasury, contributors, threats)
2. `uiStore` — UI state (active module, zoom level, selected node, panel visibility)
3. `agentStore` — AI agent conversation history and state
4. `eventStore` — event queue for visual effects (shockwaves, pulses, etc.)

2.5 Styling: Tailwind CSS + CSS Custom Properties

Tailwind for rapid layout and responsive design. **CSS Custom Properties** for the design system (all glow colors, animation timings, etc.).

We will NOT use a component library like shadcn/ui or Material UI. The entire visual identity is custom.

Using a library would make it look generic.

2.6 AI Layer: Anthropic Claude API

Why Claude over OpenAI for hackathon:

- Claude Sonnet is fast and cheap
- Excellent at structured output (JSON mode)
- Strong at analysis and summarization
- The API is simple

Usage pattern:

- All AI calls go through our Next.js API route (`/api/ai/route.ts`)
- We never expose API keys to the client
- We use streaming for the hologram assistant (text appears word by word)
- We use structured output (JSON) for data analysis (risk scores, predictions)

AI Agent Roles:

1. Governance Agent: Summarize proposals, predict outcomes, detect manipulation
2. Treasury Agent: Analyze financial health, detect anomalies, predict runway
3. Security Agent: Score wallet risk, detect sybil patterns, flag threats
4. Operations Agent: Summarize activity, suggest improvements, track KPIs

For the hackathon, we build ONE unified agent that can handle all four roles based on the context/prompt.

2.7 Blockchain Data: Alchemy SDK + Ethers.js

Alchemy SDK provides:

- `alchemy.core.getTokenBalances()` — treasury balances
- `alchemy.core.getAssetTransfers()` — transaction history (energy streams)
- `alchemy.nft.getNftsForOwner()` — NFT holdings
- `alchemy.ws` — WebSocket for real-time events

Ethers.js provides:

- Contract interaction (reading governance contracts)
- ENS resolution (wallet addresses → human names)
- ABI encoding/decoding

Data we will fetch:

- Uniswap Governor contract: proposals, vote counts, states
- Uniswap Treasury: token balances, transfer history
- Delegate registry: top delegates, voting power
- Recent transactions: for energy stream visualization

Caching strategy:

- Fetch blockchain data server-side in API routes
- Cache in Supabase with 5-minute TTL
- Client polls every 30 seconds for updates
- Mock data fallback if API fails

2.8 Database: Supabase

Tables we need:

- `proposals` — cached governance proposals
- `votes` — cached vote data
- `contributors` — contributor profiles + metrics
- `treasury_events` — treasury transaction history
- `threats` — detected security threats
- `simulations` — saved multiverse scenarios
- `user_profiles` — gamification data (XP, class, badges)
- `events` — event log for pulse animations

Why Supabase specifically:

- Postgres (real SQL, not NoSQL limitations)
- Built-in realtime subscriptions (we subscribe to new events)
- Row-level security (for multi-user support)
- Auth (wallet connect + social login)
- Instant REST API from table schema
- Free tier is generous enough for hackathon

2.9 Realtime: Supabase Realtime (Primary) + Simulated Events (Fallback)

For the hackathon demo, we have two realtime strategies:

Strategy A (Real): Supabase Realtime subscriptions. When new data is inserted into any table, all connected clients receive the update. This triggers visual events (pulses, shockwaves, new stars).

Strategy B (Demo): A simulated event emitter that fires fake events at intervals to make the demo feel alive. This is the fallback and honestly will be used during the presentation because we cannot rely on real blockchain events happening during our 3-minute demo window.

```typescript
// Simulated event emitter for demo
const DEMO_EVENTS = [
  { type: 'vote', data: { proposalId: 1, voter: '0x...', power: 50000 } },
  { type: 'transfer', data: { from: '0x...', to: '0x...', amount: 100000 } },
  { type: 'threat', data: { wallet: '0x...', riskScore: 0.87, type: 'sybil' } },
  { type: 'contributor', data: { address: '0x...', action: 'join' } },
];

let eventIndex = 0;
setInterval(() => {
  const event = DEMO_EVENTS[eventIndex % DEMO_EVENTS.length];
  eventStore.getState().pushEvent(event);
  eventIndex++;
}, 5000); // New event every 5 seconds
```

2.10 Deployment: Vercel

- Connected to GitHub repo
- Automatic deploys on push to `main`
- Preview URLs for every branch/PR
- Environment variables set in Vercel dashboard
- Edge functions for API routes (low latency)

- Analytics built in

---

3. ARCHITECTURE & SYSTEM DESIGN

3.1 High-Level Architecture

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                              CLIENT (Browser)                               │
│                                                                             │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐          │
│  │  3D Canvas       │  │  2D Overlay      │  │  State Mgmt      │          │
│  │  (R3F Scene)     │  │  (Framer M.)     │  │  (Zustand)       │          │
│  │                  │  │                  │  │                  │          │
│  │  - Universe      │  │  - HoloPanels    │  │  - daoStore      │          │
│  │  - Nodes         │  │  - Cards         │  │  - uiStore       │          │
│  │  - Streams       │  │  - Sidebar       │  │  - agentStore    │          │
│  │  - Effects       │  │  - CommandBar    │  │  - eventStore    │          │
│  └──────────┬───────┘  └──────────┬───────┘  └──────────┬───────┘          │
│             │                     │                     │                  │
│             └─────────────────────┼─────────────────────┘                  │
│                                   │                                        │
│                                   ▼                                        │
│                          ┌──────────────────┐                              │
│                          │  Event Bus       │                              │
│                          │  (Zustand        │                              │
│                          │   eventStore)    │                              │
│                          └──────────────────┘                              │
└──────────────────────────────────┬──────────────────────────────────────────┘
                                   │
                                   ▼
                          ┌──────────────────┐
                          │  Next.js API     │
                          │  Routes          │
                          │                  │
                          │  /api/ai         │
                          │  /api/blockchain │
                          │  /api/simulate   │
                          │  /api/events     │
                          └──────────┬───────┘
                                     │
                       ┌─────────────┴─────────────┐
                       │                           │
                       ▼                           ▼
                       ▼
  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
  │  Claude          │  │  Alchemy         │  │  Supabase        │
  │  API             │  │  SDK             │  │  (Postgres       │
  │                  │  │                  │  │   + Realtime)    │
  └──────────────────┘  └──────────────────┘  └──────────────────┘
```

3.2 Data Flow

Blockchain → UI Flow:

1. Alchemy SDK fetches on-chain data (proposals, balances, transfers)
2. API route processes and normalizes data
3. Data is cached in Supabase
4. Zustand `daoStore` receives normalized data
5. React components re-render with new data
6. 3D scene updates (node positions, brightness, connections)
7. Event animations trigger (pulses, shockwaves)

AI Agent Flow:

1. User types command in CommandBar
2. Command sent to `/api/ai` with current DAO context
3. Claude processes command and returns structured response
4. Response parsed: text goes to AIHologram, data goes to highlight system
5. Relevant nodes/modules are highlighted in 3D scene

Event Flow:

1. New event detected (vote, transfer, threat, etc.)
2. Event pushed to `eventStore`
3. Event listeners in 3D scene react:
    - Vote → pulse animation on governance nodes
    - Transfer → energy stream between treasury nodes
    - Threat → red anomaly node spawns
    - New contributor → shooting star animation
4. Event logged in StatusBar

3.3 Module Communication

Each module is a page route but they share state through Zustand stores. The 3D universe is always rendered (it is in the root layout). Module pages change the camera focus and the 2D overlay content.

```
User clicks "Governance Arena" in Sidebar
  → uiStore.setActiveModule('governance')
  → Camera smoothly transitions to governance cluster in 3D scene
  → 2D overlay swaps to governance panels (AnimatePresence)
```

```
  → 2D overlay swaps to governance panels (Animated reveal)
  → Relevant nodes highlight/enlarge
  → Irrelevant nodes dim
```

This creates the feeling of "flying" between modules rather than navigating pages.

---

4. COMPLETE FILE STRUCTURE

Every single file, its purpose, approximate line count, and who builds it.

```
dao-cosmos-os/
│
├── .env.local                    # Environment variables
│   # NEXT_PUBLIC_SUPABASE_URL=
│   # NEXT_PUBLIC_SUPABASE_ANON_KEY=
│   # ALCHEMY_API_KEY=
│   # ANTHROPIC_API_KEY=
│   # NEXT_PUBLIC_TARGET_DAO=uniswap
│
├── .gitignore
├── next.config.js                # Next.js config with GLSL loader
├── tailwind.config.ts            # Tailwind + custom theme
├── tsconfig.json                 # TypeScript strict mode
├── package.json                  # All dependencies
├── postcss.config.js             # PostCSS for Tailwind
│
├── public/
│   ├── fonts/
│   │   ├── Orbitron-Regular.woff2      # Display font (titles, numbers)
│   │   ├── Orbitron-Bold.woff2
│   │   ├── Exo2-Regular.woff2          # Body font (descriptions, content)
│   │   ├── Exo2-Medium.woff2
│   │   ├── Exo2-SemiBold.woff2
│   │   └── SpaceMono-Regular.woff2     # Mono font (addresses, code, data)
│   │
│   ├── textures/
│   │   ├── star-particle.png           # 16x16 soft circle for star particles
│   │   ├── noise.png                   # 512x512 Perlin noise texture
│   │   ├── nebula-blue.png             # 2048x1024 nebula background
│   │   ├── nebula-purple.png           # 2048x1024 alternate nebula
│   │   ├── grid-floor.png              # 512x512 grid texture for floor plane
│   │   ├── scan-line.png               # 4x512 horizontal scan line
│   │   └── energy-gradient.png         # 256x1 gradient for energy streams
│   │
│   ├── icons/
│   │   ├── governance.svg              # Module icon: sword/shield
```

```
│   │   ├── treasury.svg            # Module icon: reactor/energy
│   │   ├── contributors.svg        # Module icon: star cluster
│   │   ├── security.svg            # Module icon: shield/sentinel
│   │   ├── ai-agent.svg            # Module icon: hologram face
│   │   ├── simulator.svg           # Module icon: multiverse fork
│   │   └── cosmos-logo.svg         # DAO Cosmos OS logo
│   │
│   └── sounds/                     # Optional ambient audio
│       ├── ambient-hum.mp3         # Low reactor hum (looping)
│       ├── pulse.mp3               # Short pulse SFX
│       ├── alert.mp3               # Threat detection SFX
│       └── transition.mp3          # Module switch SFX
│
├── src/
│   │
│   ├── app/                        # NEXT.JS APP ROUTER
│   │   │
│   │   ├── layout.tsx              # [~80 lines] [Person A]
│   │   │   # Root layout. Wraps everything.
│   │   │   # - Imports global CSS
│   │   │   # - Renders <Providers> (Zustand, Supabase)
│   │   │   # - Renders <UniverseCanvas> (3D scene — always present)
│   │   │   # - Renders <AppShell> (sidebar, status bar, command bar)
│   │   │   # - Font loading (Orbitron, Exo 2, Space Mono)
│   │   │   # - Metadata (title, description, OG tags)
│   │   │
│   │   ├── page.tsx                # [~60 lines] [Person A]
│   │   │   # Landing / home page
│   │   │   # - "Enter the Universe" cinematic intro
│   │   │   # - Animated logo reveal
│   │   │   # - Wallet connect prompt
│   │   │   # - After connect: camera flies into the neural network
│   │   │
│   │   ├── globals.css             # [~300 lines] [Person A]
│   │   │   # Global styles
│   │   │   # - CSS custom properties (full design system)
│   │   │   # - Font face declarations
│   │   │   # - Tailwind directives
│   │   │   # - Holographic panel base styles
│   │   │   # - Glow utilities
│   │   │   # - Scrollbar styling
│   │   │   # - Keyframe animations (pulse, scan, flicker)
│   │   │
│   │   ├── providers.tsx           # [~40 lines] [Person B]
│   │   │   # Context providers
│   │   │   # - Supabase provider
│   │   │   # - Data fetching initialization
```

```
│   │   │   # - Event listener setup
│   │   │
│   │   ├── governance/
│   │   │   └── page.tsx              # [~120 lines] [Person A]
│   │   │       # Governance Arena page
│   │   │       # - Proposal battle cards grid
│   │   │       # - Active proposal detail panel
│   │   │       # - Vote power distribution chart
│   │   │       # - Whale detector panel
│   │   │       # - Governance replay controls
│   │   │
│   │   ├── treasury/
│   │   │   └── page.tsx              # [~100 lines] [Person A]
│   │   │       # Treasury Reactor page
│   │   │       # - Reactor core status panel
│   │   │       # - Energy flow Sankey diagram
│   │   │       # - Treasury health metrics
│   │   │       # - Risk overload indicators

│   │   │       # - Runway prediction display
│   │   │
│   │   ├── contributors/
│   │   │   └── page.tsx              # [~100 lines] [Person A]
│   │   │       # Contributor Galaxy page
│   │   │       # - Star system overview
│   │   │       # - Contributor detail panel (on star click)
│   │   │       # - Constellation map
│   │   │       # - Activity timeline
│   │   │       # - Reputation leaderboard
│   │   │
│   │   ├── security/
│   │   │   └── page.tsx              # [~90 lines] [Person A]
│   │   │       # Security Sentinel page
│   │   │       # - Threat map overlay
│   │   │       # - Active alerts list
│   │   │       # - Wallet risk scanner
│   │   │       # - Defense status panel
│   │   │
│   │   ├── simulator/
│   │   │   └── page.tsx              # [~100 lines] [Person A]
│   │   │       # Multiverse Simulator page
│   │   │       # - Scenario selector (market crash, governance takeover, etc.)
│   │   │       # - Timeline fork visualization controls
│   │   │       # - Timeline comparison panels
│   │   │       # - Probability meters
│   │   │
│   │   └── api/                    # BACKEND API ROUTES
│   │       │
```

```
│   │   ├── ai/
│   │   │   └── route.ts          # [~150 lines] [Person B]
│   │   │     # AI agent endpoint
│   │   │     # - Receives: { message, context, agentRole }
│   │   │     # - Builds system prompt based on agent role
│   │   │     # - Injects current DAO data as context
│   │   │     # - Streams response from Claude API
│   │   │     # - Returns: streaming text + structured data
│   │   │     # - Error handling + fallback responses
│   │   │
│   │   ├── blockchain/
│   │   │   ├── proposals/
│   │   │   │   └── route.ts      # [~120 lines] [Person B]
│   │   │   │     # Fetch governance proposals
│   │   │   │     # - Reads Uniswap Governor contract
│   │   │   │     # - Fetches proposal states, descriptions, vote counts
│   │   │   │     # - Caches in Supabase
│   │   │   │     # - Returns normalized proposal objects

│   │   │   │
│   │   │   ├── treasury/
│   │   │   │   └── route.ts      # [~100 lines] [Person B]
│   │   │   │     # Fetch treasury data
│   │   │   │     # - Token balances via Alchemy
│   │   │   │     # - Recent transfers
│   │   │   │     # - DeFi positions (if detectable)
│   │   │   │     # - Returns treasury state object
│   │   │   │
│   │   │   ├── contributors/
│   │   │   │   └── route.ts      # [~90 lines] [Person B]
│   │   │   │     # Fetch contributor data
│   │   │   │     # - Top delegates from governance
│   │   │   │     # - Activity scores (votes, proposals, transactions)
│   │   │   │     # - ENS name resolution
│   │   │   │     # - Returns contributor array
│   │   │   │
│   │   │   └── events/
│   │   │       └── route.ts      # [~60 lines] [Person B]
│   │   │         # Fetch recent on-chain events
│   │   │         # - Last 100 transactions
│   │   │         # - Vote events
│   │   │         # - Proposal state changes
│   │   │         # - Returns event stream array
│   │   │
│   │   ├── simulate/
│   │   │   └── route.ts          # [~130 lines] [Person B]
│   │   │     # Multiverse simulation endpoint
│   │   │     # - Receives: { scenario, currentState }
```

```
│   │   │   │   # - Uses Claude to generate 3 timeline predictions
│   │   │   │   # - Each timeline: narrative, metrics, probability
│   │   │   │   # - Returns: { timelines: [A, B, C] }
│   │   │   │
│   │   │   └── security/
│   │   │       └── route.ts          # [~100 lines] [Person B]
│   │   │           # Security analysis endpoint
│   │   │           # - Wallet risk scoring algorithm
│   │   │           # - Sybil detection heuristics
│   │   │           # - Anomaly detection on treasury flows
│   │   │           # - Returns: threat assessment object
│   │
│   ├── components/                   # ALL REACT COMPONENTS
│   │   │
│   │   ├── canvas/                   # 3D SCENE COMPONENTS (Person A)
│   │   │   │
│   │   │   ├── UniverseCanvas.tsx      # [~60 lines]
│   │   │   │   # Top-level R3F Canvas wrapper

│   │   │   │   # - <Canvas> with camera, renderer settings
│   │   │   │   # - <Suspense> with loading fallback
│   │   │   │   # - Post-processing effects
│   │   │   │   # - Scene lighting
│   │   │   │   # - OrbitControls
│   │   │   │
│   │   │   ├── Universe.tsx           # [~100 lines]
│   │   │   │   # Main scene graph
│   │   │   │   # - Renders all 3D sub-components
│   │   │   │   # - Manages camera transitions between modules
│   │   │   │   # - Ambient particles
│   │   │   │   # - Grid floor (faint, far below)
│   │   │   │
│   │   │   ├── StarField.tsx          # [~80 lines]
│   │   │   │   # Background star particles
│   │   │   │   # - 2000+ point particles
│   │   │   │   # - Varying sizes and brightness
│   │   │   │   # - Slow rotation for parallax
│   │   │   │   # - Uses Points + PointsMaterial
│   │   │   │   # - Star twinkle animation via shader
│   │   │   │
│   │   │   ├── NeuralNetwork.tsx       # [~200 lines]
│   │   │   │   # Central DAO brain visualization
│   │   │   │   # - Generates node positions (force-directed layout)
│   │   │   │   # - Renders NodeSphere for each entity
│   │   │   │   # - Renders connections as animated lines
│   │   │   │   # - Pulse animation when events occur
│   │   │   │   # - Cluster detection for module grouping
│   │   │   │   # - Click handlers for node selection
```

```
│   │   │   │   # - Glow intensity based on activity
│   │   │   │
│   │   │   ├── NodeSphere.tsx          # [~80 lines]
│   │   │   │   # Individual node in the network
│   │   │   │   # - Glowing sphere with custom shader
│   │   │   │   # - Color based on node type (governance=blue, treasury=gold, etc.)
│   │   │   │   # - Size based on influence/value
│   │   │   │   # - Hover effect: expand + show label
│   │   │   │   # - Click: select node, show detail panel
│   │   │   │   # - Pulse animation on activity
│   │   │   │   # - Billboard label (always faces camera)
│   │   │   │
│   │   │   ├── Connection.tsx          # [~70 lines]
│   │   │   │   # Animated connection between nodes
│   │   │   │   # - Curved line using QuadraticBezierLine
│   │   │   │   # - Energy pulse traveling along the line
│   │   │   │   # - Thickness based on relationship strength
│   │   │   │   # - Color based on type (transaction, vote, collaboration)

│   │   │   │   # - Opacity fade for inactive connections
│   │   │   │
│   │   │   ├── EnergyStream.tsx        # [~90 lines]
│   │   │   │   # Animated energy flow for treasury
│   │   │   │   # - Particle system following a path
│   │   │   │   # - Speed based on transaction velocity
│   │   │   │   # - Color: cyan for inflow, magenta for outflow
│   │   │   │   # - Trail effect (Drei <Trail>)
│   │   │   │   # - Brightness based on amount
│   │   │   │
│   │   │   ├── ReactorCore.tsx         # [~120 lines]
│   │   │   │   # Treasury reactor 3D object
│   │   │   │   # - Central glowing sphere (the "core")
│   │   │   │   # - Orbiting ring structures
│   │   │   │   # - Pulse rate = treasury activity
│   │   │   │   # - Color shifts: healthy=cyan, overloaded=red
│   │   │   │   # - Energy streams feeding into/out of core
│   │   │   │   # - Particle emissions from core
│   │   │   │
│   │   │   ├── ContributorStar.tsx     # [~70 lines]
│   │   │   │   # Individual contributor as a star
│   │   │   │   # - Point light + sprite
│   │   │   │   # - Brightness = activity level
│   │   │   │   # - Size = influence score
│   │   │   │   # - Color = contributor class
│   │   │   │   # - Hover: show name + stats
│   │   │   │   # - Constellation lines to collaborators
│   │   │   │
│   │   │   ├── ThreatNode.tsx          # [~60 lines]
```

```
│   │   │   │   # Security threat visualization
│   │   │   │   # - Red/orange glowing anomaly
│   │   │   │   # - Irregular geometry (icosahedron with noise)
│   │   │   │   # - Pulsing warning effect
│   │   │   │   # - Connected to affected nodes via red lines
│   │   │   │   # - "Virus" particle emission
│   │   │   │
│   │   │   ├── Shockwave.tsx          # [~50 lines]
│   │   │   │   # Expanding ring effect for major events
│   │   │   │   # - Torus geometry that expands and fades
│   │   │   │   # - Triggered by eventStore events
│   │   │   │   # - Color based on event type
│   │   │   │   # - Multiple can exist simultaneously
│   │   │   │
│   │   │   ├── TimelineFork.tsx        # [~100 lines]
│   │   │   │   # Multiverse timeline visualization
│   │   │   │   # - Current state at center
│   │   │   │   # - 3 branching paths diverging outward

│   │   │   │   # - Each path leads to a future state cluster
│   │   │   │   # - Animated divergence effect
│   │   │   │   # - Labels for each timeline
│   │   │   │
│   │   │   ├── CameraController.tsx    # [~80 lines]
│   │   │   │   # Smooth camera transitions
│   │   │   │   # - Listens to uiStore.activeModule
│   │   │   │   # - Lerps camera position to target
│   │   │   │   # - Configurable transition speed
│   │   │   │   # - Zoom levels (universe → cluster → node → detail)
│   │   │   │   # - Cinematic intro sequence
│   │   │   │
│   │   │   └── PostProcessing.tsx      # [~40 lines]
│   │   │       # Effect composer setup
│   │   │       # - Bloom (threshold, intensity, radius)
│   │   │       # - ChromaticAberration (subtle)
│   │   │       # - Vignette
│   │   │       # - Noise (film grain, subtle)
│   │   │
│   │   ├── ui/                        # 2D OVERLAY COMPONENTS
│   │   │   │
│   │   │   ├── AppShell.tsx            # [~60 lines] [Person A]
│   │   │   │   # Main app layout structure
│   │   │   │   # - Sidebar (left)
│   │   │   │   # - StatusBar (top)
│   │   │   │   # - CommandBar (bottom)
│   │   │   │   # - Content area (center — module pages render here)
│   │   │   │   # - Notifications (top-right)
│   │   │   │
```

```
│   │   │   ├── Sidebar.tsx          # [~90 lines] [Person A]
│   │   │   │   # Left navigation panel
│   │   │   │   # - Module list with icons
│   │   │   │   # - Active module highlight (glow)
│   │   │   │   # - Collapsed/expanded states
│   │   │   │   # - DAO health indicator
│   │   │   │   # - Wallet connection status
│   │   │   │   # - Holographic glass effect
│   │   │   │
│   │   │   ├── StatusBar.tsx         # [~70 lines] [Person A]
│   │   │   │   # Top bar with system status
│   │   │   │   # - DAO name + logo
│   │   │   │   # - System health indicator (green/yellow/red)
│   │   │   │   # - Live event feed (scrolling)
│   │   │   │   # - Active threats count
│   │   │   │   # - Treasury value (live)
│   │   │   │   # - Connected wallet display
│   │   │   │

│   │   │   ├── HoloPanel.tsx         # [~70 lines] [Person A]
│   │   │   │   # CORE COMPONENT — holographic floating panel
│   │   │   │   # - Glass blur background (backdrop-filter)
│   │   │   │   # - Cyan/blue glowing border
│   │   │   │   # - Corner accent marks
│   │   │   │   # - Subtle flicker animation
│   │   │   │   # - Float animation (Framer Motion)
│   │   │   │   # - Entrance animation (scale from 0.8 + fade)
│   │   │   │   # - Configurable: size, position, title, closable
│   │   │   │   # - Scan line overlay effect
│   │   │   │
│   │   │   ├── HoloCard.tsx          # [~60 lines] [Person A]
│   │   │   │   # Smaller holographic card (used for proposals, stats, etc.)
│   │   │   │   # - Compact version of HoloPanel
│   │   │   │   # - Hover: lift + glow intensify
│   │   │   │   # - Click: expand to detail view
│   │   │   │   # - Status indicator (active, passed, failed)
│   │   │   │   # - Mini data visualization slot
│   │   │   │
│   │   │   ├── GlowButton.tsx         # [~40 lines] [Person A]
│   │   │   │   # Sci-fi button component
│   │   │   │   # - Glowing border on hover
│   │   │   │   # - Click ripple effect (energy pulse)
│   │   │   │   # - Variants: primary (cyan), danger (red), secondary (dim)
│   │   │   │   # - Loading state: pulsing animation
│   │   │   │   # - Icon slot (left or right)
│   │   │   │
│   │   │   ├── CommandBar.tsx         # [~90 lines] [Person B]
│   │   │   │   # AI command input (bottom of screen)
```

```
│   │   │   │   # - Text input with holographic styling
│   │   │   │   # - Placeholder: "Ask the AI Commander..."
│   │   │   │   # - Command history (up/down arrows)
│   │   │   │   # - Auto-suggestions as user types
│   │   │   │   # - Submit sends to /api/ai
│   │   │   │   # - Shows streaming response in AIHologram
│   │   │   │   # - Quick action buttons: "Summarize", "Risks", "Predict"
│   │   │   │
│   │   │   ├── AIHologram.tsx          # [~100 lines] [Person B]
│   │   │   │   # AI response display
│   │   │   │   # - Floating panel with "hologram" visual treatment
│   │   │   │   # - Animated avatar (simple geometric face)
│   │   │   │   # - Streaming text display (typewriter effect)
│   │   │   │   # - When AI mentions nodes: those nodes glow in 3D scene
│   │   │   │   # - Response actions: "Show on map", "Expand", "Save"
│   │   │   │   # - Voice wave animation while "speaking"
│   │   │   │
│   │   │   ├── ProposalCard.tsx        # [~80 lines] [Person A]

│   │   │   │   # Governance proposal battle card
│   │   │   │   # - Title + summary
│   │   │   │   # - Vote bar (For vs Against — animated fill)
│   │   │   │   # - Voting power counter (animated number)
│   │   │   │   # - Status badge (Active, Passed, Defeated, Queued)
│   │   │   │   # - Impact score (AI-generated)
│   │   │   │   # - Time remaining countdown
│   │   │   │   # - "Battle card" visual theme
│   │   │   │
│   │   │   ├── TreasuryPanel.tsx       # [~80 lines] [Person A]
│   │   │   │   # Treasury statistics panel
│   │   │   │   # - Total value (animated counter)
│   │   │   │   # - Token breakdown (pie chart in holo style)
│   │   │   │   # - Inflow/outflow (24h, 7d, 30d)
│   │   │   │   # - Health status indicator
│   │   │   │   # - Runway estimate
│   │   │   │   # - Risk level meter
│   │   │   │
│   │   │   ├── ContributorProfile.tsx  # [~60 lines] [Person A]
│   │   │   │   # Individual contributor detail card
│   │   │   │   # - Avatar (generated from address)
│   │   │   │   # - ENS name or truncated address
│   │   │   │   # - Reputation score
│   │   │   │   # - Activity graph (mini sparkline)
│   │   │   │   # - Class badge (Architect, Diplomat, etc.)
│   │   │   │   # - XP level
│   │   │   │   # - Collaboration count
│   │   │   │
│   │   │   ├── ThreatAlert.tsx         # [~50 lines] [Person A]
```

```
│   │   │   │   # Security threat notification card
│   │   │   │   # - Threat type icon
│   │   │   │   # - Risk score (0-100, color-coded)
│   │   │   │   # - Affected entity
│   │   │   │   # - Description
│   │   │   │   # - Action buttons (Investigate, Dismiss)
│   │   │   │   # - Pulsing red border
│   │   │   │
│   │   │   ├── TimelinePanel.tsx       # [~70 lines] [Person A]
│   │   │   │   # Multiverse simulation result panel
│   │   │   │   # - Timeline name + probability
│   │   │   │   # - Key metrics (treasury, contributors, governance health)
│   │   │   │   # - Narrative summary
│   │   │   │   # - Visual indicator (color + icon)
│   │   │   │   # - "Explore this timeline" button
│   │   │   │
│   │   │   ├── Leaderboard.tsx         # [~60 lines] [Person B]
│   │   │   │   # Gamification leaderboard

│   │   │   │   # - Top 10 contributors by XP
│   │   │   │   # - Class distribution chart
│   │   │   │   # - Current "Raid Boss" progress
│   │   │   │   # - User's own rank
│   │   │   │
│   │   │   ├── WalletConnect.tsx       # [~50 lines] [Person B]
│   │   │   │   # Wallet connection UI
│   │   │   │   # - "Connect Wallet" button
│   │   │   │   # - Supported wallets list
│   │   │   │   # - Connected state display
│   │   │   │   # - ENS name resolution
│   │   │   │
│   │   │   ├── LoadingScreen.tsx       # [~40 lines] [Person A]
│   │   │   │   # Initial loading / cinematic intro
│   │   │   │   # - "DAO COSMOS OS" title animation
│   │   │   │   # - "Initializing neural network..." progress
│   │   │   │   # - Particle convergence effect
│   │   │   │   # - Fades out to reveal universe
│   │   │   │
│   │   │   └── Notification.tsx        # [~30 lines] [Person A]
│   │   │       # Toast notification component
│   │   │       # - Slides in from right
│   │   │       # - Types: info (cyan), warning (gold), danger (red), success (green)
│   │   │       # - Auto-dismiss after 5s
│   │   │       # - Holographic style
│   │   │
│   │   └── charts/                    # DATA VISUALIZATION COMPONENTS
│   │       │
│   │       ├── VoteBar.tsx            # [~50 lines] [Person A]
```

```
│   │   │   # Horizontal bar showing For vs Against votes
│   │   │   # - Animated fill on load
│   │   │   # - Glow effect on the winning side
│   │   │   # - Labels with vote counts
│   │   │   # - Quorum indicator line
│   │   │
│   │   ├── TreasuryFlow.tsx        # [~80 lines] [Person A]
│   │   │   # Sankey-style flow diagram
│   │   │   # - Sources on left (income)
│   │   │   # - Treasury in center
│   │   │   # - Destinations on right (spending)
│   │   │   # - Animated flow paths
│   │   │   # - Hover: highlight specific flow
│   │   │   # - Built with D3 or custom SVG
│   │   │
│   │   ├── RiskGauge.tsx           # [~40 lines] [Person A]
│   │   │   # Circular gauge meter
│   │   │   # - 0-100 risk score

│   │   │   # - Color gradient: green → yellow → red
│   │   │   # - Animated needle
│   │   │   # - Glow on high risk
│   │   │
│   │   ├── ActivitySparkline.tsx   # [~30 lines] [Person A]
│   │   │   # Mini line chart for activity over time
│   │   │   # - 30 data points
│   │   │   # - Gradient fill
│   │   │   # - No axes (just the shape)
│   │   │   # - Glow effect
│   │   │
│   │   └── TokenPieChart.tsx       # [~50 lines] [Person A]
│   │       # Treasury token distribution
│   │       # - Donut chart with holographic style
│   │       # - Animated segments
│   │       # - Labels with percentages
│   │       # - Center: total value
│   │
│   ├── three/                      # THREE.JS UTILITIES (Person A)
│   │   │
│   │   ├── shaders/
│   │   │   ├── glowVertex.glsl        # [~20 lines] Vertex shader for glow effect
│   │   │   ├── glowFragment.glsl      # [~30 lines] Fragment shader for glow effect
│   │   │   ├── pulseVertex.glsl       # [~15 lines] Vertex shader for pulse
│   │   │   ├── pulseFragment.glsl     # [~25 lines] Fragment shader with time-based pulse
│   │   │   ├── hologramVertex.glsl    # [~20 lines] Vertex shader for hologram material
│   │   │   ├── hologramFragment.glsl  # [~40 lines] Fragment: scan lines, flicker, transparency
│   │   │   ├── energyVertex.glsl      # [~15 lines] Vertex shader for energy streams
│   │   │   └── energyFragment.glsl    # [~30 lines] Fragment: flowing gradient + glow
│   │   │
```

```
│   │   │
│   │   ├── materials.ts              # [~80 lines]
│   │   │   # Custom Three.js materials
│   │   │   # - GlowMaterial (ShaderMaterial with glow shader)
│   │   │   # - PulseMaterial (animated pulse)
│   │   │   # - HologramMaterial (scan lines + transparency)
│   │   │   # - EnergyMaterial (flowing energy effect)
│   │   │   # - NodeMaterial (for network nodes, with fresnel glow)
│   │   │
│   │   ├── geometries.ts             # [~40 lines]
│   │   │   # Custom geometries
│   │   │   # - AnomalyGeometry (distorted icosahedron for threats)
│   │   │   # - ReactorRingGeometry (torus with custom UV)
│   │   │
│   │   └── utils.ts                  # [~60 lines]
│   │       # 3D utility functions
│   │       # - generateNetworkLayout() — force-directed node placement
│   │       # - lerpPosition() — smooth position interpolation

│   │       # - screenToWorld() — convert mouse to 3D coordinates
│   │       # - frustumCull() — check if point is visible
│   │       # - colorFromType() — map entity type to color
│   │
│   ├── store/                        # ZUSTAND STORES
│   │   │
│   │   ├── daoStore.ts               # [~120 lines] [Person B]
│   │   │   # DAO data store
│   │   │   # State:
│   │   │   #   proposals: Proposal[]
│   │   │   #   treasury: TreasuryState
│   │   │   #   contributors: Contributor[]
│   │   │   #   threats: Threat[]
│   │   │   #   delegates: Delegate[]
│   │   │   #   daoMeta: { name, address, chain, health }
│   │   │   # Actions:
│   │   │   #   fetchProposals()
│   │   │   #   fetchTreasury()
│   │   │   #   fetchContributors()
│   │   │   #   fetchThreats()
│   │   │   #   updateFromEvent(event)
│   │   │
│   │   ├── uiStore.ts                # [~80 lines] [Person A]
│   │   │   # UI state store
│   │   │   # State:
│   │   │   #   activeModule: 'home' | 'governance' | 'treasury' | 'contributors' | 'security' | 'simulator'
│   │   │   #   selectedNode: string | null
│   │   │   #   zoomLevel: number (0-4)
│   │   │   #   sidebarExpanded: boolean
```

```
│   │   │   # commandBarOpen: boolean
│   │   │   # modalContent: ReactNode | null
│   │   │   # cameraTarget: Vector3
│   │   │   # Actions:
│   │   │   # setActiveModule(module)
│   │   │   # selectNode(id)
│   │   │   # setZoomLevel(level)
│   │   │   # setCameraTarget(position)
│   │   │
│   │   ├── agentStore.ts          # [~70 lines] [Person B]
│   │   │   # AI agent state store
│   │   │   # State:
│   │   │   # messages: { role, content, timestamp }[]
│   │   │   # isStreaming: boolean
│   │   │   # activeAgent: 'governance' | 'treasury' | 'security' | 'operations'
│   │   │   # highlights: string[] (node IDs to highlight)
│   │   │   # Actions:
│   │   │   # sendMessage(text)

│   │   │   # clearHistory()
│   │   │   # setActiveAgent(agent)
│   │   │   # setHighlights(ids)
│   │   │
│   │   └── eventStore.ts          # [~60 lines] [Person B]
│   │       # Event bus store
│   │       # State:
│   │       # events: DAOEvent[] (last 100)
│   │       # activeEffects: Effect[] (currently animating)
│   │       # Actions:
│   │       # pushEvent(event) — adds event, triggers animations
│   │       # clearEffect(id) — removes completed animation
│   │       # startDemoMode() — fires simulated events
│   │
│   ├── services/                  # DATA SERVICES (Person B)
│   │   │
│   │   ├── blockchain.ts          # [~150 lines]
│   │   │   # Blockchain data service
│   │   │   # - Alchemy SDK initialization
│   │   │   # - fetchProposals(governorAddress) → Proposal[]
│   │   │   # - fetchTreasuryBalance(treasuryAddress) → TokenBalance[]
│   │   │   # - fetchTransferHistory(address, limit) → Transfer[]
│   │   │   # - fetchDelegates(tokenAddress) → Delegate[]
│   │   │   # - resolveENS(address) → string
│   │   │   # - subscribeToEvents(address, callback)
│   │   │   # - Error handling + retry logic
│   │   │
│   │   ├── ai.ts                 # [~100 lines]
│   │   │   # AI agent service
```

```
│   │   │   # - buildSystemPrompt(agentRole, daoContext) → string
│   │   │   # - sendMessage(message, history, agentRole) → stream
│   │   │   # - parseStructuredResponse(text) → { text, data, highlights }
│   │   │   # - Quick commands: summarize, risks, predict, explain
│   │   │
│   │   ├── supabase.ts                # [~40 lines]
│   │   │   # Supabase client
│   │   │   # - createClient()
│   │   │   # - Typed table accessors
│   │   │   # - Realtime subscription helpers
│   │   │
│   │   ├── security.ts                # [~100 lines]
│   │   │   # Security analysis service
│   │   │   # - calculateWalletRisk(address, history) → RiskScore
│   │   │   # - detectSybilPatterns(addresses) → SybilResult
│   │   │   # - detectTreasuryAnomaly(transfers) → AnomalyResult
│   │   │   # - monitorGovernanceManipulation(votes) → ManipulationResult
│   │   │

│   │   └── simulator.ts               # [~80 lines]
│   │       # Simulation engine
│   │       # - generateScenario(type, currentState) → ScenarioInput
│   │       # - runSimulation(scenario) → Timeline[]
│   │       # - calculateProbabilities(timelines) → number[]
│   │       # - formatTimelineForDisplay(timeline) → DisplayTimeline
│   │
│   ├── hooks/                        # CUSTOM HOOKS
│   │   │
│   │   ├── useDAOData.ts              # [~50 lines] [Person B]
│   │   │   # Hook to fetch and subscribe to DAO data
│   │   │   # - Fetches all data on mount
│   │   │   # - Sets up Supabase realtime subscription
│   │   │   # - Polls blockchain data every 30s
│   │   │   # - Returns: { isLoading, error, refetch }
│   │   │
│   │   ├── usePulse.ts                # [~30 lines] [Person A]
│   │   │   # Hook to trigger pulse animations
│   │   │   # - Subscribes to eventStore
│   │   │   # - Returns: { pulseNodes, pulseConnections }
│   │   │   # - Auto-clears after animation completes
│   │   │
│   │   ├── useZoom.ts                 # [~40 lines] [Person A]
│   │   │   # Hook for infinite zoom camera control
│   │   │   # - Wheel event listener
│   │   │   # - Maps zoom level to camera distance
│   │   │   # - Returns: { zoomLevel, zoomTo, resetZoom }
│   │   │
│   │   ├── useCameraTransition.ts      # [~40 lines] [Person A]
```

```
│   │   │   # Hook for smooth camera movement
│   │   │   # - Listens to uiStore.cameraTarget
│   │   │   # - Uses useFrame to lerp camera position
│   │   │   # - Configurable easing and duration
│   │   │
│   │   └── useStreamingResponse.ts     # [~50 lines] [Person B]
│   │       # Hook for streaming AI responses
│   │       # - Fetches from /api/ai with streaming
│   │       # - Returns text character by character
│   │       # - Handles errors gracefully
│   │       # - Returns: { response, isStreaming, error }
│   │
│   ├── lib/                          # UTILITIES
│   │   │
│   │   ├── constants.ts               # [~80 lines] [Person B]
│   │   │   # All constants
│   │   │   # - DAO addresses (Uniswap Governor, Treasury, Token)
│   │   │   # - Color palette (CSS variable values)

│   │   │   # - Module metadata (name, icon, description, camera position)
│   │   │   # - Animation durations
│   │   │   # - API endpoints
│   │   │   # - Gamification thresholds
│   │   │
│   │   ├── types.ts                   # [~150 lines] [Person B]
│   │   │   # TypeScript type definitions (DETAILED BELOW)
│   │   │
│   │   ├── mockData.ts                # [~200 lines] [Person B]
│   │   │   # Mock data for development and demo fallback
│   │   │   # - 5 sample proposals with vote data
│   │   │   # - Treasury state with token balances
│   │   │   # - 20 sample contributors with metrics
│   │   │   # - 3 sample threats
│   │   │   # - 10 sample events
│   │   │   # - 3 simulation timelines
│   │   │
│   │   ├── formatters.ts              # [~40 lines] [Person B]
│   │   │   # Data formatting utilities
│   │   │   # - formatAddress(addr) → "0x1234...5678"
│   │   │   # - formatUSD(amount) → "$1.23M"
│   │   │   # - formatTokenAmount(amount, decimals) → "1,234.56"
│   │   │   # - formatTimeAgo(timestamp) → "2 hours ago"
│   │   │   # - formatPercentage(value) → "67.3%"
│   │   │
│   │   └── colors.ts                  # [~30 lines] [Person A]
│   │       # Color utilities
│   │       # - Color objects for Three.js (new Color('#00f0ff'))
│   │       # - Gradient generators
```

```
│   │       # - Color interpolation for risk levels
│   │       # - Module color map
│   │
│   └── styles/
│       └── (globals.css is in app/ directory)
│
└── supabase/
    └── migrations/
        └── 001_initial.sql          # [~80 lines] [Person B]
          # Database schema (DETAILED BELOW)
```

Total estimated files: ~75 Total estimated lines: ~5,000-6,000

---

5. DATABASE SCHEMA & DATA MODELS

5.1 Supabase Tables

```sql
-- 001_initial.sql

-- Cached governance proposals
CREATE TABLE proposals (
  id TEXT PRIMARY KEY,                  -- on-chain proposal ID
  title TEXT NOT NULL,
  description TEXT,
  proposer TEXT NOT NULL,               -- wallet address
  status TEXT NOT NULL DEFAULT 'active',  -- active, passed, defeated, queued, executed
  for_votes NUMERIC NOT NULL DEFAULT 0,
  against_votes NUMERIC NOT NULL DEFAULT 0,
  abstain_votes NUMERIC NOT NULL DEFAULT 0,
  quorum NUMERIC,
  start_block BIGINT,
  end_block BIGINT,
  impact_score NUMERIC,                 -- AI-generated (0-100)
  ai_summary TEXT,                      -- AI-generated summary
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Cached vote data
CREATE TABLE votes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  proposal_id TEXT REFERENCES proposals(id),
  voter TEXT NOT NULL,                  -- wallet address
  support INTEGER NOT NULL,             -- 0=against, 1=for, 2=abstain
  weight NUMERIC NOT NULL,              -- voting power
```

```sql
  weight NUMERIC NOT NULL,              -- voting power
  block_number BIGINT,
  timestamp TIMESTAMPTZ,
  is_whale BOOLEAN DEFAULT false        -- voting power > threshold
);

-- Contributors / delegates
CREATE TABLE contributors (
  address TEXT PRIMARY KEY,
  ens_name TEXT,
  display_name TEXT,
  voting_power NUMERIC DEFAULT 0,
  delegators_count INTEGER DEFAULT 0,
  proposals_created INTEGER DEFAULT 0,
  votes_cast INTEGER DEFAULT 0,
  activity_score NUMERIC DEFAULT 0,     -- 0-100 computed metric
  reputation_score NUMERIC DEFAULT 0,   -- 0-100 reputation
  contributor_class TEXT,               -- architect, diplomat, sentinel, merchant, explorer
  xp INTEGER DEFAULT 0,
  level INTEGER DEFAULT 1,
  first_seen TIMESTAMPTZ,
  last_active TIMESTAMPTZ,
  avatar_seed TEXT                      -- for generating deterministic avatars
);

-- Treasury state snapshots
CREATE TABLE treasury_snapshots (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  timestamp TIMESTAMPTZ DEFAULT now(),
  total_value_usd NUMERIC,
  tokens JSONB,                         -- [{ symbol, balance, value_usd, address }]
  inflow_24h NUMERIC DEFAULT 0,
  outflow_24h NUMERIC DEFAULT 0,
  runway_days INTEGER,                  -- AI-estimated
  health_score NUMERIC,                 -- 0-100
  risk_level TEXT DEFAULT 'normal'      -- normal, elevated, critical
);

-- Treasury transactions
CREATE TABLE treasury_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tx_hash TEXT UNIQUE,
  from_address TEXT,
  to_address TEXT,
  token_symbol TEXT,
  amount NUMERIC,
  value_usd NUMERIC,
  event_type TEXT,                      -- transfer, swap, stake, unstake, claim
```

```sql
  timestamp TIMESTAMPTZ,
  block_number BIGINT
);

-- Security threats
CREATE TABLE threats (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  threat_type TEXT NOT NULL,            -- sybil, treasury_drain, governance_manipulation, suspicious_wallet
  severity TEXT NOT NULL DEFAULT 'medium', -- low, medium, high, critical
  risk_score NUMERIC,                   -- 0-100
  affected_entity TEXT,                 -- wallet address, proposal ID, etc.
  description TEXT,
  evidence JSONB,                       -- supporting data points
  status TEXT DEFAULT 'active',         -- active, investigating, resolved, dismissed
  detected_at TIMESTAMPTZ DEFAULT now(),
  resolved_at TIMESTAMPTZ
);


-- Multiverse simulation results
CREATE TABLE simulations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scenario_type TEXT NOT NULL,          -- market_crash, governance_takeover, treasury_depletion, whale_influence
  input_params JSONB,
  timelines JSONB,                      -- [{ name, probability, narrative, metrics }]
  created_at TIMESTAMPTZ DEFAULT now(),
  created_by TEXT                       -- wallet address
);

-- Event log for real-time animations
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type TEXT NOT NULL,             -- vote, transfer, proposal, threat, contributor_join, contributor_leave
  data JSONB,
  source_node TEXT,                     -- which node to animate from
  target_node TEXT,                     -- which node to animate to
  magnitude NUMERIC DEFAULT 1,          -- effect intensity
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Gamification profiles
CREATE TABLE user_profiles (
  wallet_address TEXT PRIMARY KEY,
  ens_name TEXT,
  class TEXT DEFAULT 'explorer',
  xp INTEGER DEFAULT 0,
  level INTEGER DEFAULT 1,
  badges JSONB DEFAULT '[]',
```

```
  achievements JSONB DEFAULT '[]',
  joined_at TIMESTAMPTZ DEFAULT now()
);

-- Enable realtime on events table (for live animations)
ALTER PUBLICATION supabase_realtime ADD TABLE events;
ALTER PUBLICATION supabase_realtime ADD TABLE threats;
ALTER PUBLICATION supabase_realtime ADD TABLE votes;
```

5.2 TypeScript Type Definitions

```typescript
// lib/types.ts

// ─── Module Types ───
export type ModuleId = 'home' | 'governance' | 'treasury' | 'contributors' | 'security' | 'simulator';


// ─── Governance Types ───
export interface Proposal {
  id: string;
  title: string;
  description: string;
  proposer: string;
  status: 'active' | 'passed' | 'defeated' | 'queued' | 'executed' | 'cancelled';
  forVotes: number;
  againstVotes: number;
  abstainVotes: number;
  quorum: number;
  startBlock: number;
  endBlock: number;
  impactScore: number;       // 0-100, AI-generated
  aiSummary: string;
  createdAt: string;
  totalVoters: number;
  topVoters: VoteRecord[];
}

export interface VoteRecord {
  voter: string;
  ensName?: string;
  support: 0 | 1 | 2;        // against, for, abstain
  weight: number;
  isWhale: boolean;
  timestamp: string;
}
```

```
// ─── Treasury Types ───
export interface TreasuryState {
  totalValueUSD: number;
  tokens: TokenBalance[];
  inflow24h: number;
  outflow24h: number;
  runwayDays: number;
  healthScore: number;        // 0-100
  riskLevel: 'normal' | 'elevated' | 'critical';
  recentTransfers: TreasuryTransfer[];
}

export interface TokenBalance {
  symbol: string;
  address: string;
  balance: number;
  valueUSD: number;
  percentage: number;         // of total treasury

  priceChange24h: number;     // percentage
}

export interface TreasuryTransfer {
  txHash: string;
  from: string;
  to: string;
  tokenSymbol: string;
  amount: number;
  valueUSD: number;
  type: 'transfer' | 'swap' | 'stake' | 'unstake' | 'claim';
  timestamp: string;
}

// ─── Contributor Types ───
export interface Contributor {
  address: string;
  ensName?: string;
  displayName: string;
  votingPower: number;
  delegatorsCount: number;
  proposalsCreated: number;
  votesCast: number;
  activityScore: number;      // 0-100
  reputationScore: number;    // 0-100
  contributorClass: ContributorClass;
  xp: number;
  level: number;
  firstSeen: string;
```

```
  firstSeen: string;
  lastActive: string;
  collaborators: string[];     // addresses of frequent collaborators
}

export type ContributorClass = 'architect' | 'diplomat' | 'sentinel' | 'merchant' | 'explorer';

// ——— Security Types ———
export interface Threat {
  id: string;
  type: 'sybil' | 'treasury_drain' | 'governance_manipulation' | 'suspicious_wallet';
  severity: 'low' | 'medium' | 'high' | 'critical';
  riskScore: number;           // 0-100
  affectedEntity: string;
  description: string;
  evidence: Record<string, any>;
  status: 'active' | 'investigating' | 'resolved' | 'dismissed';
  detectedAt: string;
}


export interface WalletRiskScore {
  address: string;
  overallRisk: number;         // 0-100
  factors: {
    age: number;               // account age risk (newer = higher risk)
    concentration: number;     // token concentration risk
    behavior: number;          // unusual behavior patterns
    connections: number;       // connections to flagged wallets
  };
  flags: string[];             // human-readable risk factors
}

// ——— Simulation Types ———
export interface SimulationScenario {
  type: 'market_crash' | 'governance_takeover' | 'treasury_depletion' | 'whale_influence' | 'custom';
  params: Record<string, any>;
  description: string;
}

export interface Timeline {
  name: string;
  probability: number;         // 0-1
  narrative: string;           // AI-generated description
  metrics: {
    treasuryValue: number;
    contributorCount: number;
    governanceHealth: number;   // 0-100
    survivalMonths: number;
```

```
  survivalMonths: number;
  };
  sentiment: 'positive' | 'neutral' | 'negative';
  keyEvents: string[];
}

// ——— Event Types ———
export interface DAOEvent {
  id: string;
  type: 'vote' | 'transfer' | 'proposal' | 'threat' | 'contributor_join' | 'contributor_leave';
  data: Record<string, any>;
  sourceNode?: string;
  targetNode?: string;
  magnitude: number;          // 0-1 (effect intensity)
  timestamp: string;
}

// ——— 3D Scene Types ———
export interface NetworkNode {

  id: string;
  type: 'governance' | 'treasury' | 'contributor' | 'contract' | 'wallet' | 'threat';
  label: string;
  position: [number, number, number]; // x, y, z
  size: number;               // 0.1 to 3
  brightness: number;         // 0 to 1
  color: string;              // hex color
  connections: string[];      // IDs of connected nodes
  metadata: Record<string, any>;
}

export interface NetworkConnection {
  source: string;             // node ID
  target: string;             // node ID
  type: 'transaction' | 'vote' | 'collaboration' | 'delegation' | 'threat';
  strength: number;           // 0-1 (line thickness)
  animated: boolean;          // whether energy pulses travel along it
}

// ——— AI Agent Types ———
export interface AgentMessage {
  role: 'user' | 'assistant';
  content: string;
  timestamp: string;
  highlights?: string[];      // node IDs to highlight
  data?: Record<string, any>;  // structured data from AI
}

export type AgentRole = 'governance' | 'treasury' | 'security' | 'operations' | 'general';
```

```
export type AgentRole = governance | treasury | security | operations | general;

// ─── Gamification Types ───
export interface UserProfile {
  walletAddress: string;
  ensName?: string;
  class: ContributorClass;
  xp: number;
  level: number;
  badges: Badge[];
  achievements: Achievement[];
}

export interface Badge {
  id: string;
  name: string;
  description: string;
  icon: string;
  earnedAt: string;

}

export interface Achievement {
  id: string;
  name: string;
  description: string;
  progress: number;          // 0-100
  completed: boolean;
}

// ─── UI State Types ───
export interface VisualEffect {
  id: string;
  type: 'shockwave' | 'pulse' | 'energy_beam' | 'star_flash' | 'alert_flash';
  origin: [number, number, number];
  color: string;
  intensity: number;
  duration: number;          // ms
  startedAt: number;         // Date.now()
}
```

---

6. UI/UX DESIGN SYSTEM — THE COSMOS DESIGN LANGUAGE

6.1 Design Philosophy

The design language is called "Cosmos". It draws from three primary inspirations:

Iron Man JARVIS — holographic floating panels, transparent UI elements, cyan glow, data scanning effects. This gives us our panel system.

Mass Effect Galaxy Map — 3D space navigation, star fields, nebula backgrounds, orbital camera. This gives us our navigation paradigm.

Ender's Game Battle Room — zero-gravity interface, 3D thinking, command-and-control aesthetic. This gives us our overall spatial metaphor.

The key principle: **Everything floats. Nothing feels "grounded."** There are no solid backgrounds, no drop shadows on white cards. Everything is semi-transparent, glowing, and slightly in motion.

6.2 Color System

```css
:root {
  /* ─── Base Colors ─── */
  --cosmos-void:        #0a0a0f;   /* Deep space black (main background) */
  --cosmos-void-light:  #12121a;   /* Slightly lighter for layering */
  --cosmos-void-lighter: #1a1a2e;  /* Panel backgrounds */


  /* ─── Primary Glow: Cyan ─── */
  --cosmos-cyan:        #00f0ff;   /* Primary accent */
  --cosmos-cyan-dim:    #00a5b0;   /* Muted cyan for secondary elements */
  --cosmos-cyan-glow:   rgba(0, 240, 255, 0.3);  /* Glow shadow */
  --cosmos-cyan-ghost:  rgba(0, 240, 255, 0.08); /* Faint background tint */

  /* ─── Alert: Magenta ─── */
  --cosmos-magenta:     #ff2d6a;   /* Danger / threat / negative */
  --cosmos-magenta-dim: #b01e4a;
  --cosmos-magenta-glow: rgba(255, 45, 106, 0.3);

  /* ─── Gold: Achievement ─── */
  --cosmos-gold:        #ffd700;   /* Treasury / achievement / XP */
  --cosmos-gold-dim:    #b09500;
  --cosmos-gold-glow:   rgba(255, 215, 0, 0.3);

  /* ─── Purple: Mystic ─── */
  --cosmos-purple:      #7b2ff7;   /* Simulation / AI / governance */
  --cosmos-purple-dim:  #5a1fb5;
  --cosmos-purple-glow: rgba(123, 47, 247, 0.3);

  /* ─── Green: Positive ─── */
  --cosmos-green:       #00ff88;   /* Success / healthy / positive */
  --cosmos-green-dim:   #00b060;
  --cosmos-green-glow:  rgba(0, 255, 136, 0.3);
```

```
  /* ─── Text Colors ─── */
  --cosmos-text-primary:  #e0e8ff;    /* Main text (slightly blue white) */
  --cosmos-text-secondary: #7a8ba8;   /* Muted text */
  --cosmos-text-dim:      #3a4560;    /* Very dim text (labels, timestamps) */

  /* ─── Glass Effect ─── */
  --cosmos-glass-bg:      rgba(10, 10, 20, 0.7);   /* Panel background */
  --cosmos-glass-border:  rgba(0, 240, 255, 0.15);  /* Default border */
  --cosmos-glass-blur:    12px;       /* Backdrop blur amount */

  /* ─── Module-Specific Colors ─── */
  --module-governance:    var(--cosmos-purple);
  --module-treasury:      var(--cosmos-gold);
  --module-contributors:  var(--cosmos-cyan);
  --module-security:      var(--cosmos-magenta);
  --module-simulator:     var(--cosmos-green);
  --module-ai:            var(--cosmos-purple);

  /* ─── Animation Timings ─── */
  --transition-fast:      150ms;

  --transition-normal:    300ms;
  --transition-slow:      600ms;
  --transition-cinematic: 1200ms;

  /* ─── Typography ─── */
  --font-display:         'Orbitron', monospace;      /* Titles, headings, numbers */
  --font-body:            'Exo 2', sans-serif;        /* Body text, descriptions */
  --font-mono:            'Space Mono', monospace;    /* Addresses, data, code */
}
```

6.3 Typography Scale

```
css

/* Titles — Orbitron */
.cosmos-title-xl    { font-family: var(--font-display); font-size: 2.5rem; font-weight: 700; letter-spacing: 0.15em; text-trans
.cosmos-title-lg    { font-family: var(--font-display); font-size: 1.75rem; font-weight: 700; letter-spacing: 0.1em; text-trans
.cosmos-title-md    { font-family: var(--font-display); font-size: 1.25rem; font-weight: 600; letter-spacing: 0.08em; }
.cosmos-title-sm    { font-family: var(--font-display); font-size: 0.875rem; font-weight: 600; letter-spacing: 0.12em; text-t

/* Body — Exo 2 */
.cosmos-body-lg     { font-family: var(--font-body); font-size: 1.125rem; font-weight: 400; line-height: 1.6; }
.cosmos-body-md     { font-family: var(--font-body); font-size: 0.9375rem; font-weight: 400; line-height: 1.5; }
.cosmos-body-sm     { font-family: var(--font-body); font-size: 0.8125rem; font-weight: 400; line-height: 1.4; }

/* Data — Space Mono */
.cosmos-data-lg     { font-family: var(--font-mono); font-size: 1.5rem; font-weight: 400; letter-spacing: 0.05em; }
```

```
.cosmos-data-md    { font-family: var(--font-mono); font-size: 1rem; font-weight: 400; letter-spacing: 0.03em; }
.cosmos-data-sm    { font-family: var(--font-mono); font-size: 0.75rem; font-weight: 400; letter-spacing: 0.05em; }
```

6.4 Core Component Specs

HoloPanel (The Foundation Component)

This is the most important component. Every piece of 2D UI uses this as its base.

```
┌──────────────────────────────────────────┐
│ ┌──                                ──┐   │ ← Corner accents (4px cyan lines)
│                                        │
│ PANEL TITLE                      [×]   │ ← Orbitron font, uppercase, tracked
│ ──────────────────────────────────────── │ ← 1px cyan line separator
│                                        │
│ Content area                           │
│ Glass blur background                  │
│ Semi-transparent                       │
│                                        │
│ └──                                ──┘   │ ← Corner accents
└──────────────────────────────────────────┘
```

Visual properties:

- Background: `var(--cosmos-glass-bg)` — rgba(10, 10, 20, 0.7)
- Backdrop filter: `blur(12px) saturate(150%)`
- Border: `1px solid var(--cosmos-glass-border)`
- Corner accents: 4 small L-shaped elements in each corner, `var(--cosmos-cyan)` with 2px width
- Subtle scan line overlay (repeating linear gradient, 2px transparent, 1px rgba(0,240,255,0.03))
- Entrance animation: scale from 0.95 to 1, opacity 0 to 1, 300ms ease-out
- Idle animation: very subtle float (translateY oscillating ±2px over 4s)
- Hover state: border brightens to `rgba(0, 240, 255, 0.4)`, slight scale(1.01)
- Box shadow: `0 0 30px var(--cosmos-cyan-glow), inset 0 0 30px rgba(0, 240, 255, 0.05)`

GlowButton

```
┌──────────────────┐
│ BUTTON TEXT      │ ← Orbitron font, uppercase, small
└──────────────────┘
```

Visual properties:

- Background: transparent

- Border: 1px solid `var(--cosmos-cyan-dim)`
- Text: `var(--cosmos-text-primary)`
- Padding: 8px 20px
- Letter spacing: 0.15em
- Hover: border → `var(--cosmos-cyan)`, text → white, box-shadow glow appears
- Active: background briefly flashes `var(--cosmos-cyan-ghost)`, scale(0.98)
- Variants:
  - Primary: cyan border and glow
  - Danger: magenta border and glow
  - Gold: gold border and glow
  - Ghost: very dim border, minimal glow

StatusBar (Top)

```
◉ DAO COSMOS OS  | ● SYSTEM HEALTHY | ⚡ $2.3B TVL | 🔗 0x12..34 |
```

- Full width, fixed top, z-index above 3D scene
- Height: 48px
- Background: `linear-gradient(180deg, rgba(10,10,20,0.9), rgba(10,10,20,0.6))`
- Bottom border: `1px solid var(--cosmos-glass-border)`
- Content: flex row, space-between
- Logo + DAO name on left (Orbitron)
- Health status, TVL, wallet on right (Space Mono for values)
- Event ticker scrolling in center (latest events)

Sidebar (Left)

```
┌──┐
│ ≡ │ ← Toggle button
└──┘
┌──┐
│🧠│ ← Neural Network (Home)
│⚔️│ ← Governance Arena
│⚡│ ← Treasury Reactor
│⭐│ ← Contributor Galaxy
│🛡️│ ← Security Sentinel
│🤖│ ← AI Command
│🌌│ ← Simulator
└──┘
```

- Fixed left, z-index above 3D scene
- Collapsed: 60px wide, icons only
- Expanded: 240px wide, icons + labels
- Background: same glass effect as HoloPanel
- Active module: icon glows brightly, left cyan accent bar
- Hover: icon brightens, label slides in
- Transition: 300ms ease-out

CommandBar (Bottom)

- Fixed bottom, full width
- Height: 56px (expandable to 200px when AI responds)
- Glass background
- Input field: transparent, Exo 2 font, placeholder text dims
- Send button: GlowButton variant
- Quick action chips above input: "Summarize", "Risks", "Predict", "Explain"
- When AI responds: panel expands upward, AIHologram appears

6.5 Responsive Strategy

For hackathon, we optimize for:

1. **16:9 desktop** (primary — judges' laptops)
2. **16:10 MacBook** (secondary)
3. Tablet and mobile are NOT supported (no time, judges won't check)

Breakpoints:

- Default: 1280px+ (full layout)
- `@media (max-width: 1279px)`: Collapse sidebar permanently, reduce panel sizes
- Below 1024px: Not supported, show "Desktop required" message

6.6 Keyframe Animations

6.6 Keyframe Animations

```css
/* Scan line moving down a panel */
@keyframes scanLine {
  0% { transform: translateY(-100%); }
  100% { transform: translateY(100%); }
}

/* Gentle pulse for glowing elements */
@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.7; }
}

/* Stronger pulse for alerts */
@keyframes alertPulse {
  0%, 100% { box-shadow: 0 0 10px var(--cosmos-magenta-glow); }
  50% { box-shadow: 0 0 30px var(--cosmos-magenta-glow), 0 0 60px var(--cosmos-magenta-glow); }
}

/* Flicker effect for holographic elements */
@keyframes flicker {
  0%, 97%, 100% { opacity: 1; }
  98% { opacity: 0.8; }
  99% { opacity: 0.9; }
}
```

```
/* Number counting up */
@keyframes countUp {
  from { --count: 0; }
  to { --count: var(--target); }
}

/* Rotating ring for reactor */
@keyframes rotateRing {
  from { transform: rotateZ(0deg); }
  to { transform: rotateZ(360deg); }
}

/* Energy stream flow */
@keyframes energyFlow {
  from { stroke-dashoffset: 100; }
  to { stroke-dashoffset: 0; }
}
```

6.7 Interaction Patterns

Node Selection:

1. User hovers over 3D node → node expands 20%, label appears (Billboard component)
2. User clicks node → node pulses, HoloPanel slides in from right with node details
3. User clicks elsewhere → panel exits, node returns to normal

Module Navigation:

1. User clicks module in Sidebar → sidebar icon glows
2. Camera smoothly flies to module's region in 3D space (1.2s ease-in-out)
3. 2D overlay panels fade out (200ms), new module panels fade in (200ms, staggered)
4. 3D nodes for that module brighten, other nodes dim

AI Interaction:

1. User clicks CommandBar or presses / (keyboard shortcut)
2. CommandBar activates (border brightens, placeholder changes)
3. User types question and presses Enter
4. "Analyzing..." animation in AIHologram panel
5. Response streams in character by character (typewriter effect)
6. If response mentions specific entities, those nodes pulse in 3D scene
7. User can click "Show on map" to fly camera to relevant nodes

Event Reactions (passive):

1. New vote event arrives → governance nodes pulse blue
2. Large transfer → energy beam shoots between treasury nodes
3. New threat → red anomaly node spawns with alert sound, ThreatAlert slides in
4. New contributor → shooting star enters contributor galaxy cluster

---

7. MODULE 1: DAO NEURAL NETWORK (Central Hub)

7.1 Overview

This is the default view — what users see when they first enter. The entire DAO is visualized as a 3D neural brain where nodes represent entities and connections represent relationships.

7.2 Node Types and Visual Rules

Node Type	Shape	Base Color	Size Range	Brightness Rule
Governance (proposals)	Octahedron	#7b2ff7 (purple)	0.3 - 1.5	Voting activity
Treasury (wallets)	Sphere	#ffd700 (gold)	0.5 - 2.0	Balance size
Contributor (delegates)	Icosahedron	#00f0ff (cyan)	0.2 - 1.0	Activity score
Contract (smart contracts)	Box	#00ff88 (green)	0.3 - 0.8	Interaction count
Threat (anomalies)	Irregular icosahedron	#ff2d6a (magenta)	0.4 - 1.2	Risk score

7.3 Connection Types

Connection Type	Color	Thickness	Animation
Transaction	Gold gradient	Proportional to amount	Energy pulse travels along
Vote	Purple	Proportional to voting power	Pulse on new votes
Collaboration	Cyan	Proportional to frequency	Gentle glow
Delegation	Light blue	Proportional to delegated power	Continuous dim pulse
Threat	Red (dashed)	Fixed 1px	Rapid flicker

7.4 Layout Algorithm

We use a simplified force-directed layout computed on initial data load:

```typescript
function generateNetworkLayout(nodes: NetworkNode[]): void {
  // Step 1: Assign clusters based on module
  const clusters = {
    governance: new Vector3(-5, 2, 0),
    treasury: new Vector3(5, 2, 0),
    contributors: new Vector3(0, -3, -2),
    security: new Vector3(0, 4, 3),
    contracts: new Vector3(0, 0, 0),   // center
  };

  // Step 2: Position each node near its cluster with random offset
  nodes.forEach(node => {
    const clusterCenter = clusters[node.type] || clusters.contracts;
    const offset = new Vector3(
      (Math.random() - 0.5) * 4,
      (Math.random() - 0.5) * 4,
      (Math.random() - 0.5) * 4,
    );
    node.position = clusterCenter.clone().add(offset).toArray();
  });

  // Step 3: Simple repulsion pass (prevent overlap)
  for (let i = 0; i < 50; i++) {
    nodes.forEach((a, ai) => {
      nodes.forEach((b, bi) => {
        if (ai >= bi) return;
        const dist = new Vector3(...a.position).distanceTo(new Vector3(...b.position));
        if (dist < 1.5) {
          const repulsion = new Vector3(...a.position).sub(new Vector3(...b.position)).normalize().multiplyScalar(0.1);
          a.position = new Vector3(...a.position).add(repulsion).toArray();
          b.position = new Vector3(...b.position).sub(repulsion).toArray();
        }
      });
    });
  }
}
```

7.5 Interaction Details

Hover on node:

- Node scale: 1.0 → 1.3 (200ms ease-out)
- Node emissive intensity: current → current + 0.5
- Billboard label appears above node (fade in 150ms)

- Connected nodes brighten slightly
- Connected edges highlight

Click on node:

- Camera smoothly moves closer to node (800ms)
- Node emits shockwave ring
- Detail HoloPanel slides in from right
- Panel content depends on node type:
  - Governance node: proposal details, vote bar, AI summary
  - Treasury node: wallet balance, recent transactions, token breakdown
  - Contributor node: profile, stats, reputation, collaborators
  - Threat node: risk analysis, evidence, recommended actions

Zoom levels:

- Level 0 (furthest): Entire universe. Nodes appear as points of light. Labels hidden.
- Level 1: Cluster level. Node shapes visible. Cluster labels visible.
- Level 2: Node level. Individual labels visible. Connections visible.
- Level 3: Detail level. Node internal detail visible. All metadata shown.
- Level 4 (closest): Single node fills view. Full data panel.

---

8. MODULE 2: GOVERNANCE ARENA

8.1 Overview

Governance becomes a strategy battle system. Proposals are battle cards. Votes are energy flows. Whale voters are giant units entering the battlefield.

8.2 3D Scene (Governance Region)

When the user navigates to Governance Arena:

- Camera flies to the governance cluster (upper-left quadrant of the neural network)
- Governance nodes enlarge and spread out
- Each proposal node becomes a full "battle card" floating in 3D space
- Non-governance nodes dim to 20% opacity
- A faint circular arena floor appears beneath the proposals
- "Battle" ambiance: subtle energy crackling between proposal nodes

8.3 2D Overlay — Proposal Cards

The main 2D content is a grid of ProposalCards displayed in HoloPanels.

Layout:

- Grid of proposal cards (2 columns, scrollable)
- Featured/active proposal in larger panel (if one is currently voting)
- Vote power distribution chart on right side
- Whale detector in bottom-right

ProposalCard detailed spec:

```
┌──────────────────────────────────────────────────────────────┐
│ ┌──                                                  ──┐     │
│                                                              │
│ PROPOSAL #127                                ⚔      │ ← Proposal ID + battle icon
│ ──────────────────────────────────────────────────────────── │
│                                                              │
│ "Upgrade fee switch to 1/5..."                      │ ← Title (Exo 2, 2 lines max)
│                                                              │
│ ┌──────────────────────────────────────────────────────┐     │
│ │                                                      │     │ ← Vote bar (green=for, red=against)
│ │ 67.3% FOR                    32.7% AGAINST │     │
│ └──────────────────────────────────────────────────────┘     │
│                                                              │
│ VOTING POWER: 12.4M UNI                             │ ← Space Mono, animated counter
│ VOTERS: 847                                          │
│ QUORUM:                                    78%       │ ← Quorum progress bar
│                                                              │
│ Impact: ●●●●●●●●●○○ 8/10                            │ ← AI-generated impact score
│ Status: ● ACTIVE                                     │ ← Status badge (green dot)
│ Ends in: 2d 14h 33m                                  │ ← Countdown (Orbitron, small)
│                                                              │
│ └──                                                ──┘     │
└──────────────────────────────────────────────────────────────┘
```

Interaction on click → Expanded view:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ PROPOSAL #127 — GOVERNANCE BATTLE                                            │
│ ──────────────────────────────────────────────────────────────────────────── │
│                                                                              │
│ "Upgrade fee switch to 1/5 protocol fee"                                     │
│                                                                              │
│ AI SUMMARY:                                                                  │
```

```
│ "This proposal would direct 20% of trading fees to the  │
│ DAO treasury, estimated at $40M annually. Supporters    │
│ argue it funds development; opponents fear LP exodus."  │
│                                                         │
│ ┌── VOTE BATTLEFIELD ──────────────────────────────────┐
│ │                                                      │
│ │ FOR (67.3%)        ●●● vs ●●      AGAINST (32.7%)   │
│ │ ▓▓▓
```

8.5 Governance Replay

Users can replay historical proposals like a match replay:

- Timeline scrubber at bottom of panel
- Play/pause/speed controls
- As timeline progresses, votes appear chronologically
- Vote bar animates in real-time
- Can see exact moment whales entered
- Can see how momentum shifted

Implementation: Store vote timestamps, replay by iterating through sorted votes with delays.

---

9. MODULE 3: TREASURY REACTOR

9.1 Overview

The DAO treasury becomes a fusion energy reactor. Money flows are energy streams. The reactor's health represents the treasury's health.

9.2 3D Scene (Treasury Region)

When navigating to Treasury Reactor:

- Camera flies to treasury cluster (upper-right quadrant)
- Central ReactorCore object becomes prominent
- Orbiting ring structures activate (start spinning)
- Energy streams become visible (animated particle flows)
- Non-treasury nodes dim
- Ambient particles increase around reactor
- Subtle "reactor hum" increases

ReactorCore 3D Object:

- Center: large glowing sphere (radius 1.5)

    - Custom shader: animated fresnel glow, internal "plasma" movement
    - Color: normally `var(--cosmos-gold)` with cyan highlights
    - Overloaded: shifts to red/orange
    - Depleting: dims significantly

- Orbit rings: 3 torus geometries at different angles
  - Rotate at different speeds
  - Emit faint particle trails
  - Width proportional to treasury diversity
- Energy streams: particle systems following bezier paths
  - Inflow streams: cyan particles flowing toward core
  - Outflow streams: gold particles flowing away from core
  - Stream width proportional to transaction amount
  - Stream speed proportional to transaction velocity

9.3 2D Overlay — Treasury Panels

Main treasury panel:

```
┌──────────────────────────────────────────────────────────┐
│ REACTOR CORE STATUS                                      │
│ ──────────────────────────────────────────────────────── │
│                                                          │
│ TOTAL VALUE                                              │
│ $2,341,567,890                    │ ← Orbitron, large, gold glow
│ ▲ +2.3% (24h)                     │ ← Green for positive
│                                                          │
│ ┌──── TOKEN COMPOSITION ──────────────────────┐          │
│ │                                             │          │
│ │ ▓▓▓▓▓ UNI   $1.8B   (77%)                   │          │
│ │ ▓▓▓▓  USDC  $340M   (15%)                   │          │
│ │ ▓▓    ETH   $180M   (7.7%)                  │          │
│ │ ▓     Other $21M    (0.9%)                  │          │
│ │                                             │          │
│ └─────────────────────────────────────────────┘          │
│                                                          │
│ ENERGY FLOW (24h)                                        │
│ Inflow:  ▲ $12.4M                 │ ← Cyan
│ Outflow: ▼ $8.7M                  │ ← Gold
│ Net:     ▲ $3.7M                  │ ← Green
│                                                          │
│ HEALTH: ▓▓▓▓▓
```

The reactor visually "overheats" when risk increases:

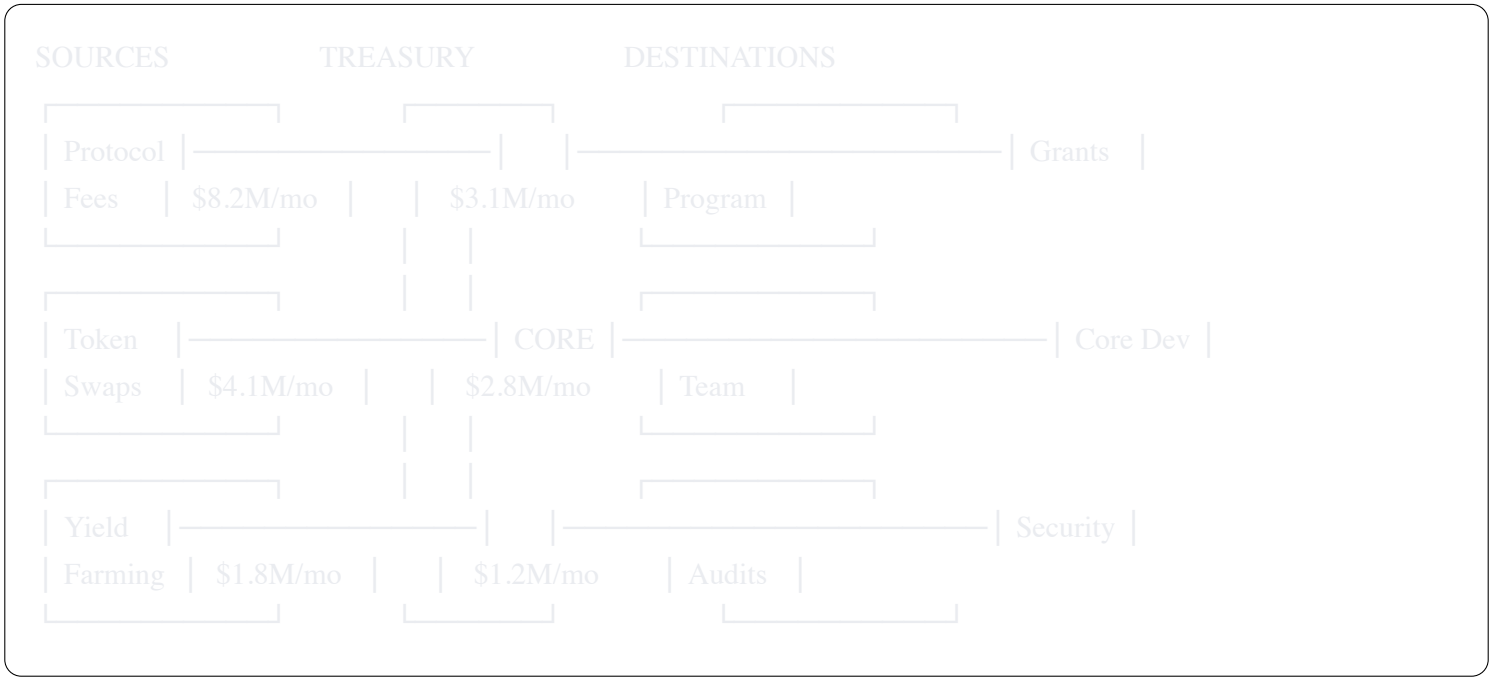
Risk Level	Visual Effect
Normal (0-30)	Calm golden glow, slow ring rotation
Elevated (31-60)	Orange tint, faster rotation, particle increase
High (61-80)	Red tint, rapid rotation, warning particles, screen edge glow red
Critical (81-100)	Intense red pulsing, rings wobble, alarm particles, screen shake

Risk factors that increase the score:

- Over 80% of treasury in a single token
- Large sudden outflows (>5% of total in 24h)
- Unusual transfer patterns
- Smart contract interaction with unknown addresses
- Token price volatility
- Low runway prediction

9.5 Treasury Flow Visualization

A Sankey diagram rendered in the holographic style:



Rendered as animated flowing lines (not static). Energy particles travel along each path. Width proportional to amount. Color: cyan for inflow, gold for outflow.

---

10. MODULE 4: CONTRIBUTOR GALAXY

10. MODULE 4: CONTRIBUTOR GALAXY

10.1 Overview

Every contributor is a star. Their brightness represents activity. Their size represents influence. Collaborations form constellations.

10.2 3D Scene

When navigating to Contributor Galaxy:

- Camera flies to contributor cluster (lower quadrant)
- Stars spread out to fill more of the viewport
- Star field behind becomes denser
- Constellation lines appear between collaborating stars
- Nebula background intensifies (purple/blue gradient)
- Camera slowly rotates around the galaxy for ambient effect

ContributorStar 3D Object:

- Base: Point sprite with custom star texture
- Size: 0.1 (inactive) → 1.0 (most active)
- Brightness: 0.2 (faded) → 1.0 (fully active)
- Color: based on contributor class:
  - Architect (builder): cyan #00f0ff
  - Diplomat (governance): purple #7b2ff7
  - Sentinel (security): magenta #ff2d6a
  - Merchant (treasury): gold #ffd700
  - Explorer (community): green #00ff88
- Hover: star grows 2x, rays appear, name label shows
- Click: camera zooms to star, detail panel opens

Constellation lines:

- Thin white/cyan lines between frequent collaborators
- Opacity based on collaboration frequency
- Form visible "shapes" for teams/workstreams
- Labeled ("Core Dev", "Grants Committee", "Security Team")

Dynamic events:

- New contributor: shooting star animation entering the galaxy
- Contributor goes inactive (30+ days): star slowly fades to dim red

- Contributor goes inactive (30+ days): star slowly fades to dim red
- Collaboration event: brief bright line flash between two stars

10.3 2D Overlay

Galaxy overview panel:

```
┌─────────────────────────────────────┐
│ CONTRIBUTOR GALAXY                  │
│ ─────────────────────────────────── │
│                                     │
│ TOTAL CONTRIBUTORS: 1,247           │
│ ACTIVE (30d): 384                   │
│ NEW THIS WEEK: 23 ⭐                │
│                                     │
│ CLASS DISTRIBUTION:                 │
│ Architect ▓▓▓
```

COLLABORATORS:
a16z.eth, gauntlet.eth, +12 more

BADGES: 🏆📦⚡🛡️

---

11. MODULE 5: SECURITY SENTINEL

11.1 Overview

The DAO's immune system. Threats appear as anomalies attacking the network. Defense systems appear as security drones.

11.2 3D Scene

When navigating to Security:

- Camera flies to security region (top quadrant)
- All threat nodes become prominent (enlarged, pulsing red)
- Red connection lines appear showing threat relationships
- A faint "shield" hemisphere appears around the neural network
- "Defense drones" (small floating geometric shapes) patrol the perimeter
- Ambient particles turn slightly red-tinged

ThreatNode 3D Object:

- Base: Distorted icosahedron (vertices displaced with noise)
- Color: #ff2d6a (magenta/red)
- Emission: high (self-illuminating)
- Animation: vertices constantly shifting (organic "virus" feel)
- Particle emission: small red particles floating outward
- Pulse: rapid aggressive pulse (0.5s cycle)
- Connection to affected nodes: dashed red lines

11.3 2D Overlay

Threat dashboard:

🛡️ SECURITY SENTINEL

```
│ ──────────────────────────────────────────────────────────────── │
│                                                                  │
│ SYSTEM STATUS: ● ELEVATED                                        │ ← Yellow status
│ ACTIVE THREATS: 3                                                │
│ RESOLVED (30d): 12                                               │
│                                                                  │
│ ┌─── ACTIVE THREATS ──────────────────────────────────────────┐ │
│ │                                                              │ │
│ │ 🔴 CRITICAL — Sybil Attack                                   │ │
│ │ Risk: 87/100                                                 │ │
│ │ Cluster of 47 wallets with                                   │ │
│ │ identical voting patterns                                    │ │
│ │ [INVESTIGATE]  [DISMISS]                                     │ │
│ │                                                              │ │
│ │ 🟠 HIGH — Whale Movement                                     │ │
│ │ Risk: 72/100                                                 │ │
│ │ 0x7a3f... moved 5M UNI to                                    │ │
│ │ unknown address                                              │ │
│ │ [INVESTIGATE]  [DISMISS]                                     │ │
│ │                                                              │ │
│ │ 🟡 MEDIUM — Unusual Pattern                                  │ │
│ │ Risk: 45/100                                                 │ │
│ │ Treasury outflow spike:                                      │ │
│ │ 3x normal in last 6 hours                                    │ │
│ │ [INVESTIGATE]  [DISMISS]                                     │ │
│ │                                                              │ │
│ └──────────────────────────────────────────────────────────────┘ │
│                                                                  │
│ WALLET RISK SCANNER                                              │
│ Enter address: [_____________]                                   │
│ [SCAN]                                                           │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

11.4 Wallet Risk Scoring Algorithm

```typescript
function calculateWalletRisk(address: string, history: Transfer[]): WalletRiskScore {
  const factors = {
    age: 0,
```

```
  concentration: 0,
  behavior: 0,
  connections: 0,
};

// Age risk: newer wallets are higher risk
const firstTx = history[history.length - 1]?.timestamp;
const ageDays = firstTx ? daysSince(firstTx) : 0;
factors.age = ageDays < 7 ? 90 : ageDays < 30 ? 70 : ageDays < 90 ? 40 : ageDays < 365 ? 20 : 5;

// Concentration: large % of single token
const totalValue = history.reduce((sum, tx) => sum + tx.valueUSD, 0);
const maxSingleTx = Math.max(...history.map(tx => tx.valueUSD));
factors.concentration = Math.min(100, (maxSingleTx / totalValue) * 100);

// Behavior: unusual patterns
const txPerDay = history.length / Math.max(ageDays, 1);
if (txPerDay > 50) factors.behavior += 40;    // Very high frequency
const uniqueCounterparties = new Set(history.map(tx => tx.to)).size;
if (uniqueCounterparties < 3 && history.length > 10) factors.behavior += 30; // Few counterparties
factors.behavior = Math.min(100, factors.behavior);

// Connections: interactions with flagged wallets (simplified)
// In production, check against a blacklist. For hackathon, use heuristics.
factors.connections = 0; // Placeholder

// Overall risk: weighted average
const overallRisk = Math.round(
  factors.age * 0.2 +
  factors.concentration * 0.3 +
  factors.behavior * 0.35 +
  factors.connections * 0.15
);

const flags: string[] = [];
if (factors.age > 60) flags.push('New wallet (< 30 days)');
if (factors.concentration > 70) flags.push('High token concentration');
if (factors.behavior > 50) flags.push('Unusual transaction patterns');
if (factors.connections > 30) flags.push('Connected to flagged wallets');

return { address, overallRisk, factors, flags };
}
```

---

12. MODULE 6: AI COMMAND AGENTS

12.1 System Prompt Architecture

12.1 System Prompt Architecture

The AI agent uses role-specific system prompts that include current DAO data as context.

```typescript
function buildSystemPrompt(role: AgentRole, daoContext: any): string {
  const base = `You are the AI Commander of DAO Cosmos OS, an advanced AI assistant
for decentralized autonomous organization management. You speak concisely and
authoritatively, like a starship AI. Keep responses under 150 words unless the
user asks for detail. When you identify specific entities (wallets, proposals,
contributors), return their IDs so the 3D interface can highlight them.

Current DAO: ${daoContext.daoMeta.name}
Chain: Ethereum Mainnet
Treasury Value: $${formatUSD(daoContext.treasury.totalValueUSD)}
Active Proposals: ${daoContext.proposals.filter(p => p.status === 'active').length}
Total Contributors: ${daoContext.contributors.length}
Active Threats: ${daoContext.threats.filter(t => t.status === 'active').length}

Always respond with JSON in this format:
{
  "text": "Your natural language response",
  "highlights": ["node_id_1", "node_id_2"],
  "riskLevel": "low|medium|high|critical",
  "data": { any structured data relevant to the query }
}`;

  const rolePrompts = {
    governance: `\n\nYou are the GOVERNANCE AGENT. You specialize in proposal analysis,
vote prediction, and governance manipulation detection.

CURRENT PROPOSALS:
${JSON.stringify(daoContext.proposals.slice(0, 5), null, 2)}

TOP DELEGATES:
${JSON.stringify(daoContext.contributors.slice(0, 10).map(c => ({
  name: c.displayName, votingPower: c.votingPower, votes: c.votesCast
})), null, 2)}`,

    treasury: `\n\nYou are the TREASURY AGENT. You specialize in financial analysis,
risk assessment, and treasury optimization.

TREASURY STATE:
${JSON.stringify(daoContext.treasury, null, 2)}`,

    security: `\n\nYou are the SECURITY AGENT. You specialize in threat detection,
wallet risk analysis, and sybil detection.
```

```
ACTIVE THREATS:
${JSON.stringify(daoContext.threats, null, 2)}`,

    operations: `\n\nYou are the OPERATIONS AGENT. You provide activity summaries,

contributor insights, and DAO health assessments.

CONTRIBUTOR STATS:
Active: ${daoContext.contributors.filter(c => c.activityScore > 50).length}
Inactive: ${daoContext.contributors.filter(c => c.activityScore <= 50).length}
New this week: ${daoContext.contributors.filter(c => isThisWeek(c.firstSeen)).length}`,

    general: '', // Use base prompt only
  };

  return base + (rolePrompts[role] || '');
}
```

12.2 AI Command Quick Actions

Pre-built commands that users can trigger with one click:

Command	AI Role	What It Does
"Summarize"	Operations	30-second summary of DAO state
"Risks"	Security	Top 3 threats with risk scores
"Predict [proposal]"	Governance	Vote outcome prediction
"Health check"	Treasury	Financial health assessment
"Who is [address]?"	Operations	Contributor profile analysis
"Compare proposals"	Governance	Side-by-side proposal analysis
"Simulate [scenario]"	Operations	Trigger multiverse simulation

12.3 AIHologram Visual Design

```
┌──────────────────────────────────────────┐
│                                          │
│    ┌─────────┐                           │
│    │ ◉ ◉ ◉   │          ← Geometric "face" (3 dots)
│    │ ╲ ___ ╱ │            Animated: dots pulse when "speaking"
│    └─────────┘                           │
│                                          │
│    ┌──────────────────────────────────┐  │
```

```
│ │ Analysis complete, Commander.  │ │ ← Typewriter text effect
│ │ Treasury health is at 87%.     │ │
│ │ Primary risk: 77% concentration │ │
│ │ in UNI tokens. Recommend       │ │

│ │ diversification strategy.      │ │
│ │                                │ │
│ │ Highlighting treasury nodes... │ │
│ └────────────────────────────────┘ │
│                                    │
│ ┌──────┐ ┌──────┐ ┌──────────┐    │
│ │ EXPAND │ │ MAP  │ │ NEW QUERY │  │ ← Action buttons
│ └──────┘ └──────┘ └──────────┘    │
└────────────────────────────────────┘
```

- Panel floats near bottom-right of screen
- Gentle float animation
- Voice wave visualization: thin horizontal line that oscillates when AI is "speaking"
- Geometric face: 3 circles arranged in triangle, connected by lines. Dots pulse during response streaming. Subtle rotation animation.

---

13. MODULE 7: MULTIVERSE DAO SIMULATOR

13.1 Overview

The most ambitious feature. Users select a scenario. The AI generates 3 possible future timelines. The 3D scene shows the DAO splitting into parallel universes.

13.2 Scenarios

Scenario	Input Parameters	What AI Simulates
Market Crash	Crash severity (25%, 50%, 75%)	Impact on treasury value, contributor exodus, governance paralysis
Governance Takeover	Attacker budget ($1M, $10M, $100M)	Token accumulation, vote manipulation, proposal outcomes
Treasury Depletion	Burn rate multiplier (2x, 5x, 10x)	Runway remaining, forced cutbacks, protocol death spiral
Whale Influence	Whale size, alignment (hostile/friendly)	Governance shift, community reaction, protocol direction

Custom	User-defined parameters	AI interprets and simulates

13.3 3D Timeline Fork Visualization

When simulation runs:

1. Current DAO neural network freezes in place
2. Three translucent copies begin to separate outward (fork animation)
3. Each copy shifts in a different direction (120° apart)
4. Connecting "timeline threads" link current state to each future
5. Each timeline slowly evolves:
    - Timeline A (best case): nodes brighten, new connections form
    - Timeline B (neutral): some changes, mixed signals
    - Timeline C (worst case): nodes dim, connections break, threat nodes multiply

Visual:

```
                Timeline A (Positive)
               /
Current State ——— Timeline B (Neutral)
               \
                Timeline C (Negative)
```

Each timeline is a mini neural network rendered at 50% scale with appropriate color tinting:

- Positive: green tint, bright
- Neutral: blue tint, normal
- Negative: red tint, dim with broken connections

13.4 2D Overlay

Scenario selector:

```
┌──────────────────────────────────────┐
│ 🌌 MULTIVERSE SIMULATOR              │
│ ──────────────────────────────────── │
│                                      │
│ SELECT SCENARIO:                     │
│                                      │
│ ┌──────────┐ ┌──────────────┐        │
│ │ 📉 MARKET │ │ ⚔️ GOV       │        │
│ │  CRASH   │ │  TAKEOVER    │        │
```

```
│                                          │
│  ┌──────────────┐  ┌──────────────┐      │
│  │ 💰 TREAS.    │  │ 🐋 WHALE     │      │
│  │ DEPLETION    │  │ INFLUENCE    │      │
│  └──────────────┘  └──────────────┘      │
│                                          │
│                                          │
│  SEVERITY: [──────●──────]  50%          │ ← Slider
│                                          │
│  [🚀 SIMULATE]                           │
│                                          │
└──────────────────────────────────────────┘
```

Results (after simulation):

Three TimelinePanels side by side:

```
┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
│ TIMELINE A       │ │ TIMELINE B       │ │ TIMELINE C       │
│ "Resilient"      │ │ "Stagnant"       │ │ "Collapse"       │
│                  │ │                  │ │                  │
│ Prob: 35%        │ │ Prob: 45%        │ │ Prob: 20%        │
│                  │ │                  │ │                  │
│ Treasury:        │ │ Treasury:        │ │ Treasury:        │
│ $1.8B ▼22%       │ │ $1.2B ▼48%       │ │ $400M ▼83%       │
│                  │ │                  │ │                  │
│ Contributors:    │ │ Contributors:    │ │ Contributors:    │
│ 1,100 ▼12%       │ │ 800 ▼36%         │ │ 200 ▼84%         │
│                  │ │                  │ │                  │
│ Gov Health:      │ │ Gov Health:      │ │ Gov Health:      │
│ 72/100           │ │ 45/100           │ │ 15/100           │
│                  │ │                  │ │                  │
│ Survival:        │ │ Survival:        │ │ Survival:        │
│ 3.5 years        │ │ 1.2 years        │ │ 4 months         │
│                  │ │                  │ │                  │
│ [EXPLORE]        │ │ [EXPLORE]        │ │ [EXPLORE]        │
└──────────────────┘ └──────────────────┘ └──────────────────┘
```

---

14. GAMIFICATION SYSTEM

14.1 XP Calculation

```typescript
```

```
const XP_REWARDS = {
  vote_cast: 50,          // Per vote
  proposal_created: 500,  // Per proposal
  delegation_received: 10, // Per delegator
  security_report: 200,   // Per valid threat report
  treasury_management: 100, // Per treasury action
  collaboration: 25,      // Per collaboration event
  streak_bonus: {         // Consecutive days active
    3: 50,
    7: 150,
    30: 500,
  },
};
```

14.2 Class Assignment

Classes are assigned based on primary activity:

```typescript
function assignClass(contributor: Contributor): ContributorClass {
  const activities = {
    architect: contributor.proposalsCreated * 5 + contributor.collaborations,
    diplomat: contributor.votesCast * 2 + contributor.delegatorsCount,
    sentinel: contributor.securityReports * 10,
    merchant: contributor.treasuryActions * 5,
    explorer: contributor.communityEngagement * 3,
  };

  return Object.entries(activities)
    .sort(([, a], [, b]) => b - a)[0][0] as ContributorClass;
}
```

14.3 Level Thresholds

Level	XP Required	Title
1-5	0-500	Cadet
6-15	500-2,500	Ensign

16-30	2,500-10,000	Lieutenant
31-50	10,000-25,000	Commander
51-75	25,000-75,000	Captain
76-100	75,000-200,000	Admiral

14.4 Raid Boss System

Community-wide objectives appear as "boss fights":

```
┌─────────────────────────────────────────┐
│ 🐉 RAID BOSS: TREASURY MILESTONE        │
│ ─────────────────────────────────────── │
│                                         │
│ OBJECTIVE: Grow treasury to $3B         │
│                                         │
│ ▓▓▓
```

```
// PostProcessing.tsx
import { EffectComposer, Bloom, ChromaticAberration, Vignette, Noise } from '@react-three/postprocessing';
import { BlendFunction, KernelSize } from 'postprocessing';

export function PostProcessing() {
  return (
    <EffectComposer>
      <Bloom
        intensity={1.5}
        luminanceThreshold={0.2}
        luminanceSmoothing={0.9}
        kernelSize={KernelSize.LARGE}
      />
      <ChromaticAberration
        offset={[0.0005, 0.0005]}
        radialModulation={true}
        modulationOffset={0.5}
      />
      <Vignette
        eskil={false}
        offset={0.1}
        darkness={0.8}
      />
      <Noise
        premultiply
        blendFunction={BlendFunction.SOFT_LIGHT}
        opacity={0.3}
      />
    </EffectComposer>
  );
}
```

15.2 Custom Glow Shader

```glsl
// glowVertex.glsl
varying vec3 vNormal;
varying vec3 vPosition;

void main() {
  vNormal = normalize(normalMatrix * normal);
  vPosition = (modelViewMatrix * vec4(position, 1.0)).xyz;
  gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
}

// glowFragment.glsl
uniform vec3 glowColor;
uniform float intensity;
uniform float falloff;
varying vec3 vNormal;
varying vec3 vPosition;

void main() {
  vec3 viewDir = normalize(-vPosition);
  float fresnel = pow(1.0 - abs(dot(viewDir, vNormal)), falloff);
  vec3 color = glowColor * fresnel * intensity;
  gl_FragColor = vec4(color, fresnel * 0.8);
}
```

15.3 Pulse Animation Shader

```glsl
// pulseFragment.glsl
uniform vec3 baseColor;
uniform float time;
uniform float pulseSpeed;
uniform float pulseIntensity;
```

```
varying vec3 vNormal;
varying vec3 vPosition;

void main() {
  float pulse = sin(time * pulseSpeed) * 0.5 + 0.5;
  vec3 viewDir = normalize(-vPosition);
  float fresnel = pow(1.0 - abs(dot(viewDir, vNormal)), 2.0);

  vec3 color = baseColor * (1.0 + pulse * pulseIntensity);
  float alpha = fresnel * (0.6 + pulse * 0.4);

  gl_FragColor = vec4(color, alpha);
}
```

15.4 Performance Budget

Element	Max Count	Technique
Network nodes	200	Instanced meshes (`<Instances>`)
Connections	500	Instanced lines or `<Line>` with limited points
Star particles	3,000	Points geometry (single draw call)
Energy particles	500	Points geometry with custom shader
Post-processing	4 effects	Minimal kernel sizes
Textures	<50MB total	Compressed, power-of-2 dimensions

Target: 60 FPS on M1 MacBook Air (judges' likely hardware)

15.5 Shockwave Effect Implementation

```typescript
```

```
// Shockwave.tsx
function Shockwave({ origin, color, duration = 1500 }: ShockwaveProps) {
  const meshRef = useRef<THREE.Mesh>(null);
  const startTime = useRef(Date.now());

  useFrame(() => {
    if (!meshRef.current) return;
    const elapsed = Date.now() - startTime.current;
    const progress = elapsed / duration;

    if (progress >= 1) {
      meshRef.current.visible = false;
      return;
    }

    // Expand ring
    const scale = progress * 10;
    meshRef.current.scale.set(scale, scale, scale);

    // Fade out
    const material = meshRef.current.material as THREE.MeshBasicMaterial;
    material.opacity = 1 - progress;
  });

  return (
    <mesh ref={meshRef} position={origin} rotation={[Math.PI / 2, 0, 0]}>
      <torusGeometry args={[1, 0.02, 8, 64]} />
      <meshBasicMaterial color={color} transparent opacity={1} side={THREE.DoubleSide} />
    </mesh>
  );
}
```

---

16. BLOCKCHAIN INTEGRATION LAYER

16.1 Alchemy SDK Configuration

```typescript
// services/blockchain.ts
import { Alchemy, Network, AssetTransfersCategory } from 'alchemy-sdk';

const alchemy = new Alchemy({
```

```
  apiKey: process.env.ALCHEMY_API_KEY,
  network: Network.ETH_MAINNET,
});

// Uniswap contract addresses
const CONTRACTS = {
  governor: '0x408ED6354d4973f66138C91495F2f2FCbd8724C3',   // Uniswap Governor Bravo
  timelock: '0x1a9C8182C09F50C8318d769245beA52c32BE35BC',   // Timelock
  token: '0x1f9840a85d5aF5bf1D1762F925BDADdC4201F984',      // UNI token
  treasury: '0x1a9C8182C09F50C8318d769245beA52c32BE35BC',   // Treasury (same as timelock)
};
```

16.2 Fetching Proposals

```typescript
import { ethers } from 'ethers';

const GOVERNOR_ABI = [
  'function proposalCount() view returns (uint256)',
  'function proposals(uint256) view returns (uint256 id, address proposer, uint256 eta, uint256 startBlock, uint256 endBlo
  'function state(uint256) view returns (uint8)',
  'event ProposalCreated(uint256 id, address proposer, address[] targets, uint256[] values, string[] signatures, bytes[] calld
];

async function fetchProposals(): Promise<Proposal[]> {
  const provider = new ethers.AlchemyProvider('mainnet', process.env.ALCHEMY_API_KEY);
  const governor = new ethers.Contract(CONTRACTS.governor, GOVERNOR_ABI, provider);

  // Get proposal created events (last 1000 blocks for hackathon speed)
  const filter = governor.filters.ProposalCreated();
  const events = await governor.queryFilter(filter, -100000);

  const proposals: Proposal[] = [];

  for (const event of events.slice(-10)) { // Last 10 proposals
    const args = event.args!;
    const id = args.id.toString();
    const state = await governor.state(id);

    const stateMap: Record<number, string> = {
      0: 'pending', 1: 'active', 2: 'cancelled', 3: 'defeated',
      4: 'passed', 5: 'queued', 6: 'expired', 7: 'executed',
    };

    const proposalData = await governor.proposals(id);

    proposals.push({
```

```
    proposals.push({
      id,
      title: extractTitle(args.description), // First line of description
      description: args.description,
      proposer: args.proposer,
      status: stateMap[state] || 'unknown',
      forVotes: Number(ethers.formatUnits(proposalData.forVotes, 18)),
      againstVotes: Number(ethers.formatUnits(proposalData.againstVotes, 18)),
      abstainVotes: Number(ethers.formatUnits(proposalData.abstainVotes, 18)),
      startBlock: Number(proposalData.startBlock),
      endBlock: Number(proposalData.endBlock),
      quorum: 40000000, // 40M UNI
      impactScore: 0, // Will be filled by AI
      aiSummary: '', // Will be filled by AI
      createdAt: new Date().toISOString(),
      totalVoters: 0,
      topVoters: [],

    });
  }

  return proposals;
}
```

16.3 Fetching Treasury Balance

```typescript
async function fetchTreasuryBalance(): Promise<TreasuryState> {
  // Get ETH balance
  const ethBalance = await alchemy.core.getBalance(CONTRACTS.treasury);

  // Get token balances
  const tokenBalances = await alchemy.core.getTokenBalances(CONTRACTS.treasury);

  // Get recent transfers
  const transfers = await alchemy.core.getAssetTransfers({
    fromAddress: CONTRACTS.treasury,
    category: [AssetTransfersCategory.ERC20, AssetTransfersCategory.EXTERNAL],
    maxCount: 50,
    order: 'desc',
  });

  const inboundTransfers = await alchemy.core.getAssetTransfers({
    toAddress: CONTRACTS.treasury,
    category: [AssetTransfersCategory.ERC20, AssetTransfersCategory.EXTERNAL],
    maxCount: 50,
    order: 'desc',
```

```
  });

  // Process and return (simplified for hackathon)
  return {
    totalValueUSD: 2340000000, // Would need price API for real calculation
    tokens: processTokenBalances(tokenBalances),
    inflow24h: calculateRecentFlow(inboundTransfers.transfers, 24),
    outflow24h: calculateRecentFlow(transfers.transfers, 24),
    runwayDays: 0, // AI will estimate
    healthScore: 87,
    riskLevel: 'normal',
    recentTransfers: formatTransfers([...transfers.transfers, ...inboundTransfers.transfers]),
  };
}
```

---

17. AI INTEGRATION LAYER

17.1 API Route Implementation

typescript

```
// app/api/ai/route.ts
import Anthropic from '@anthropic-ai/sdk';
import { NextRequest } from 'next/server';

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY,
});

export async function POST(req: NextRequest) {
  const { message, context, agentRole } = await req.json();

  const systemPrompt = buildSystemPrompt(agentRole || 'general', context);

  const stream = await anthropic.messages.stream({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 1000,
    system: systemPrompt,
    messages: [{ role: 'user', content: message }],
  });

  // Return streaming response
  const encoder = new TextEncoder();
  const readable = new ReadableStream({
    async start(controller) {
      for await (const event of stream) {
```

```
      if (event.type === 'content_block_delta' && event.delta.type === 'text_delta') {
        controller.enqueue(encoder.encode(event.delta.text));
      }
    }
    controller.close();
  },
});

return new Response(readable, {
  headers: {
    'Content-Type': 'text/plain; charset=utf-8',
    'Transfer-Encoding': 'chunked',
  },
});
}
```

17.2 Client-Side Streaming Hook

```typescript
// hooks/useStreamingResponse.ts
export function useStreamingResponse() {
  const [response, setResponse] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const sendMessage = async (message: string, context: any, agentRole: AgentRole) => {
    setIsStreaming(true);
    setResponse('');
    setError(null);

    try {
      const res = await fetch('/api/ai', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message, context, agentRole }),
      });

      const reader = res.body?.getReader();
      const decoder = new TextDecoder();

      if (!reader) throw new Error('No reader available');

      let fullResponse = '';
      while (true) {
        const { done, value } = await reader.read();
        if (done) break;
```

```
      const chunk = decoder.decode(value);
      fullResponse += chunk;
      setResponse(fullResponse);
    }
  } catch (err) {
    setError(err instanceof Error ? err.message : 'AI request failed');
  } finally {
    setIsStreaming(false);
  }
};

return { response, isStreaming, error, sendMessage };
}
```

---

18. REALTIME EVENT SYSTEM

18.1 Event Architecture

typescript

```
// store/eventStore.ts
import { create } from 'zustand';

interface EventStore {
  events: DAOEvent[];
  activeEffects: VisualEffect[];
  demoMode: boolean;

  pushEvent: (event: DAOEvent) => void;
  clearEffect: (id: string) => void;
  startDemoMode: () => void;
  stopDemoMode: () => void;
}

export const useEventStore = create<EventStore>((set, get) => ({
  events: [],
  activeEffects: [],
  demoMode: false,

  pushEvent: (event) => {
    // Add to event log
    set(state => ({
      events: [event, ...state.events].slice(0, 100), // Keep last 100
    }));
```

```
    // Generate visual effect based on event type
    const effect = eventToEffect(event);
    if (effect) {
      set(state => ({
        activeEffects: [...state.activeEffects, effect],
      }));

      // Auto-remove effect after duration
      setTimeout(() => {
        get().clearEffect(effect.id);
      }, effect.duration);
    }
  },

  clearEffect: (id) => {
    set(state => ({
      activeEffects: state.activeEffects.filter(e => e.id !== id),
    }));
  },

  startDemoMode: () => {
    set({ demoMode: true });
    runDemoEventLoop();
  },

  stopDemoMode: () => {
    set({ demoMode: false });
  },
}));

function eventToEffect(event: DAOEvent): VisualEffect | null {
  switch (event.type) {
    case 'vote':
      return {
        id: crypto.randomUUID(),
        type: 'pulse',
        origin: [0, 2, 0], // governance cluster center
        color: '#7b2ff7',
        intensity: event.magnitude,
        duration: 1500,
        startedAt: Date.now(),
      };
    case 'transfer':
      return {
        id: crypto.randomUUID(),
        type: 'energy_beam',
        origin: [5, 2, 0], // treasury cluster center
        color: '#ffd700',
```

```
      intensity: event.magnitude,
      duration: 2000,
      startedAt: Date.now(),
    };
  case 'threat':
    return {
      id: crypto.randomUUID(),
      type: 'alert_flash',
      origin: [0, 4, 3], // security cluster
      color: '#ff2d6a',
      intensity: 1.0,
      duration: 3000,
      startedAt: Date.now(),
    };
  case 'contributor_join':
    return {
      id: crypto.randomUUID(),

      type: 'star_flash',
      origin: [
        (Math.random() - 0.5) * 8,
        -3 + (Math.random() - 0.5) * 4,
        -2 + (Math.random() - 0.5) * 4,
      ],
      color: '#00f0ff',
      intensity: 0.8,
      duration: 2500,
      startedAt: Date.now(),
    };
  default:
    return null;
 }
}

// Demo event loop
let demoInterval: NodeJS.Timeout;
function runDemoEventLoop() {
 const store = useEventStore.getState();

 const demoEvents: DAOEvent[] = [
  { id: '1', type: 'vote', data: { proposal: '#127', voter: 'vitalik.eth', power: 50000 }, magnitude: 0.5, timestamp: new Date(
  { id: '2', type: 'transfer', data: { from: '0x1a9c...', to: '0x7a3f...', amount: 500000, token: 'UNI' }, magnitude: 0.7, timesta
  { id: '3', type: 'threat', data: { type: 'sybil', severity: 'high', wallets: 47 }, magnitude: 0.9, timestamp: new Date().toISOS
  { id: '4', type: 'vote', data: { proposal: '#127', voter: 'a16z.eth', power: 2100000 }, magnitude: 1.0, timestamp: new Date
  { id: '5', type: 'contributor_join', data: { address: '0xnew...', ens: 'newstar.eth' }, magnitude: 0.4, timestamp: new Date(),
  { id: '6', type: 'transfer', data: { from: 'protocol', to: '0x1a9c...', amount: 8200000, token: 'USDC' }, magnitude: 0.8, time
 ];
```

```
  let index = 0;
  demoInterval = setInterval(() => {
    if (!store.demoMode) {
      clearInterval(demoInterval);
      return;
    }
    store.pushEvent({ ...demoEvents[index % demoEvents.length], id: crypto.randomUUID() });
    index++;
  }, 4000);
}
```

---

19. COMPLETE API ROUTES

19.1 Route Summary

Route	Method	Purpose	Person
`/api/ai`	POST	AI agent commands	B
`/api/blockchain/proposals`	GET	Fetch governance proposals	B
`/api/blockchain/treasury`	GET	Fetch treasury state	B
`/api/blockchain/contributors`	GET	Fetch contributor data	B
`/api/blockchain/events`	GET	Fetch recent on-chain events	B
`/api/simulate`	POST	Run multiverse simulation	B
`/api/security`	POST	Security analysis	B

---

20. AUTHENTICATION & WALLET CONNECTION

For the hackathon, authentication is simplified:

Option A (Recommended for speed): No auth. The app works without wallet connection. Display mock data. If wallet is connected, show personalized data.

Option B (If time allows): RainbowKit / ConnectKit for wallet connection. This gives us the connected wallet address. We use it to show the user's own voting power, XP, and class.

```typescript
// Simplified wallet connection (no library needed for demo)
```

```
async function connectWallet(): Promise<string | null> {
  if (typeof window.ethereum === 'undefined') return null;

  try {
    const accounts = await window.ethereum.request({
      method: 'eth_requestAccounts',
    });
    return accounts[0];
  } catch {
    return null;
  }
}
```

---

21. STATE MANAGEMENT ARCHITECTURE

21.1 Store Relationship Diagram

```
daoStore (data)
  ↓ provides data to
  ├── All UI components (read)
  ├── AI service (context for prompts)
  └── eventStore (data changes trigger events)

uiStore (navigation & UI)
  ↓ controls
  ├── CameraController (camera position)
  ├── Sidebar (active module highlight)
  ├── Module pages (which page renders)
  └── NeuralNetwork (node highlighting/dimming)

agentStore (AI)
  ↓ connects
  ├── CommandBar (input)
  ├── AIHologram (output)
  └── NeuralNetwork (highlight nodes from AI response)

eventStore (events)
  ↓ triggers
  ├── Shockwave effects
  ├── Pulse animations
  ├── StatusBar event ticker
  └── Notification toasts
```

---

22. PERFORMANCE OPTIMIZATION

22.1 Critical Optimizations

1. **Instanced Meshes:** All nodes use `<Instances>` from Drei. 200 nodes = 1 draw call instead of 200.
2. **Frustum Culling:** R3F handles this automatically, but ensure all meshes have proper bounding spheres.
3. **Level of Detail:** At zoom level 0-1, render nodes as simple points. Only render full geometry at zoom level 2+.
4. **Memoization:** Every 3D component wraps expensive calculations in `useMemo`. Node positions, colors, sizes — all memoized.
5. **Texture Atlasing:** Star particle, noise texture, and scan line texture all packed into one atlas texture.
6. **Lazy Loading:** Module pages use `next/dynamic` for code splitting. Contributor Galaxy code only loads when navigated to.
7. **Debounced Updates:** Blockchain data fetches are debounced to prevent re-render storms. Only update Zustand store when data actually changes.
8. **Canvas Element Caching:** Use `<Html occlude>` sparingly. Too many HTML elements inside Canvas kills performance.

---

23. DEPLOYMENT & DEVOPS

23.1 Vercel Deployment

```bash
# Install Vercel CLI
npm i -g vercel

# Login
vercel login

# Deploy (first time — sets up project)
vercel

# Deploy to production
vercel --prod
```

23.2 Environment Variables (Vercel Dashboard)

```
ALCHEMY_API_KEY=your_alchemy_key
ANTHROPIC_API_KEY=your_anthropic_key
```

```
ANTHROPIC_API_KEY=your_anthropic_key
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_key
NEXT_PUBLIC_TARGET_DAO=uniswap
```

23.3 Pre-Deploy Checklist

- ☐ All API keys set in Vercel environment variables
- ☐ Mock data fallback works when APIs fail
- ☐ Build succeeds locally (`npm run build`)
- ☐ No TypeScript errors
- ☐ 3D scene loads within 3 seconds
- ☐ Demo mode works (simulated events)
- ☐ Mobile shows "Desktop required" message
- ☐ Lighthouse performance score > 70

---

24. DIVISION OF WORK — HOUR BY HOUR

Assuming 48-hour hackathon (Friday 6PM → Sunday 6PM)

FRIDAY (Hours 0-8)

6:00 PM — Kickoff

Person A (Visuals)	Person B (Data)
Clone repo, verify boilerplate works	Clone repo, set up all API keys
Install all 3D dependencies	Set up Supabase project, create all tables
Test R3F renders a basic scene	Test Alchemy API returns data

7:00 PM

Person A	Person B
Build `globals.css` — entire CSS design system	Write `types.ts` — all TypeScript interfaces
Define all CSS variables, keyframes, fonts	Write `constants.ts` — all addresses, colors, module metadata
Build Tailwind config	Write `mockData.ts` — complete mock dataset

8:00 PM

Person A	Person B
Build `UniverseCanvas.tsx` + `PostProcessing.tsx`	Build `blockchain.ts` service — Alchemy wrapper
Camera setup, lighting, bloom effect	Fetch proposals, treasury, contributors
Star field background particles	Cache results in Supabase

9:00 PM

Person A	Person B
Build `StarField.tsx` — 3000 star particles	Build `/api/blockchain/proposals/route.ts`
Twinkle animation, parallax rotation	Build `/api/blockchain/treasury/route.ts`
	Build `/api/blockchain/contributors/route.ts`

10:00 PM

Person A	Person B
Build `NodeSphere.tsx` with glow shader	Build `daoStore.ts` — Zustand store
Hover effect, click handler, billboard label	Fetch actions, mock data fallback
Color/size mapping from data	Wire API routes to store

11:00 PM

Person A	Person B
Build `Connection.tsx` — animated lines	Build `/api/ai/route.ts` — Claude proxy
Energy pulse traveling along connections	Streaming response setup
QuadraticBezierLine from Drei	System prompt architecture

12:00 AM

Person A	Person B
Build `NeuralNetwork.tsx` — combine nodes + connections	Build `agentStore.ts`
Force-directed layout algorithm	Build `useStreamingResponse.ts` hook
Module clustering	Test AI responses with DAO context

Module clustering	Test AI responses with DAO context

1:00 AM

Person A	Person B
Build `HoloPanel.tsx` — THE core UI component	Build `CommandBar.tsx` — AI input
Glass blur, glow border, corner accents	Quick action buttons
Scan line overlay, float animation	Command history

FRIDAY CHECKPOINT (1:30 AM): ✅ 3D universe with neural network visible ✅ HoloPanel component working ✅ Real blockchain data flowing (or mock data) ✅ AI agent responding to commands → **GO TO SLEEP** (sleep until 8:00 AM = 6.5 hours)

SATURDAY (Hours 14-32)

8:00 AM — Day 2 Start

Person A	Person B
Build `GlowButton.tsx`, `StatusBar.tsx`	Build `AIHologram.tsx` — response display
Build `Sidebar.tsx` with module navigation	Typewriter effect, voice wave animation
Build `AppShell.tsx` — layout wrapper	Wire AI highlights to 3D node system

9:00 AM

Person A	Person B
Build `CameraController.tsx`	Build `eventStore.ts`
Smooth transitions between modules	Event → visual effect mapping
Lerp camera position on module change	Demo event loop

10:00 AM

Person A	Person B
Build `Shockwave.tsx` effect	Build `security.ts` service
Expanding ring on major events	Wallet risk scoring algorithm

Expanding ring on major events	Wallet risk scoring algorithm
Multiple simultaneous effects	Sybil detection heuristics

11:00 AM — GOVERNANCE ARENA SPRINT

Person A	Person B
Build `ProposalCard.tsx`	Build vote visualization data pipeline
Battle card design, vote bar, countdown	Process vote events from blockchain
Hover/click interactions	Calculate whale threshold

12:00 PM

Person A	Person B
Build `governance/page.tsx`	Build `/api/security/route.ts`
Card grid layout, active proposal panel	Security analysis endpoint
Wire ProposalCards to daoStore	Anomaly detection

1:00 PM

Person A	Person B
Build `VoteBar.tsx` chart component	Build `/api/simulate/route.ts`
Animated fill, glow on winning side	Multiverse simulation via Claude
Quorum indicator	Timeline generation

2:00 PM — TREASURY REACTOR SPRINT

Person A	Person B
Build `ReactorCore.tsx` — 3D reactor	Build `simulator.ts` service
Pulsing sphere, orbiting rings	Scenario generation
Color shifts based on health	Probability calculation

3:00 PM

Person A	Person B
Build `EnergyStream.tsx` — particle flows	Build gamification logic
Inflow/outflow visualization	XP calculation
Trail effects	Class assignment
	Level thresholds

4:00 PM

Person A	Person B
Build `TreasuryPanel.tsx`	Build `Leaderboard.tsx`
Treasury stats in holographic panel	Top contributors
Token composition chart	Class distribution

5:00 PM

Person A	Person B
Build `treasury/page.tsx`	Wire security data into threat display
Reactor view, energy flows, panels	Build `ThreatAlert.tsx` data
Risk gauge integration	

6:00 PM

Person A	Person B
Build `RiskGauge.tsx`	Build `security/page.tsx` data integration
Circular gauge with color gradient	Wire threats to 3D ThreatNode
Animated needle	Wallet scanner input

7:00 PM — CONTRIBUTOR GALAXY SPRINT

Person A	Person B
Build `ContributorStar.tsx`	Build ENS resolution for top contributors

Star sprite with brightness/size rules	Build collaboration detection
Color by class	

8:00 PM

Person A	Person B
Build constellation lines	Integration work — make sure all stores
Build `contributors/page.tsx`	are connected, data flows correctly
Galaxy overview panel	Fix any API issues

9:00 PM

Person A	Person B
Build `ContributorProfile.tsx`	Build demo mode — verify all simulated
Detail card on star click	events trigger correct visuals
Activity sparkline	Test all quick commands

10:00 PM — SIMULATOR SPRINT

Person A	Person B
Build `TimelineFork.tsx` — 3D fork effect	Build simulation API integration
Current state → 3 branching paths	Wire scenario selector to `/api/simulate`
Animated divergence	

11:00 PM

Person A	Person B
Build `TimelinePanel.tsx`	Build `simulator/page.tsx` integration
Result display for each timeline	Wire simulation results to 3D + panels
Metrics comparison	

SATURDAY CHECKPOINT (12:00 AM): ✅ All 5 modules navigable ✅ Real data in governance + treasury ✅ AI agent working with context ✅ Multiverse simulator produces results ✅ Security threats display ✅ Contributor galaxy shows stars → **GO TO SLEEP** (sleep until 7:00 AM = 7 hours)

SUNDAY (Hours 37-48)

7:00 AM — Polish Day

Person A	Person B
Build `LoadingScreen.tsx` cinematic intro	Deploy first version to Vercel
"DAO COSMOS OS" title animation	Verify all env vars set
Particle convergence effect	Test production build

8:00 AM

Person A	Person B
Polish shockwave effects	Polish AI responses
Add event animations for votes, transfers	Tune system prompts for better output
Make whale detection dramatic	Add error handling everywhere

9:00 AM

Person A	Person B
Polish module transitions	Wire demo mode button in StatusBar
Smooth camera flights between modules	Make mock data more realistic
Ensure node dimming/highlighting works	

10:00 AM

Person A	Person B
Polish holographic panels	End-to-end testing
Ensure scan lines, flicker, float all work	Test all API routes
Color consistency check	Test AI with various questions

11:00 AM

Person A	Person B
Add ambient touches:	Deploy FINAL version to Vercel
- Subtle screen edge glow on alerts	Record backup video of demo
- Number animations (counting up)	(Screen record full walkthrough)
- Hover effects on all interactive elements	

12:00 PM — DEMO PREPARATION

Both people together:

- Write demo script (see Section 25)
- Assign speaking parts
- Rehearsal #1 (practice full 3-minute demo)
- Identify awkward transitions
- Fix any bugs discovered during rehearsal

1:00 PM

- Rehearsal #2
- Tune timing
- Pre-load specific screens for demo
- Ensure demo mode events fire at good intervals
- Prepare for "what if" scenarios (what if API is slow? what if 3D doesn't load?)

2:00 PM

- Rehearsal #3 (final)
- Prepare backup plan: pre-recorded video
- Double-check Vercel deployment is live
- Clean up any console errors
- Prepare answer for judge questions

3:00-6:00 PM — PRESENTATION

Execute the rehearsed demo. Win.

---

25. DEMO SCRIPT & PRESENTATION STRATEGY

25.1 The 3-Minute Demo Script

Opening [0:00 - 0:20] — THE HOOK

Speaker A stands at the podium. Screen shows blank.

> "Every DAO today is managed through dashboards and spreadsheets. Governance proposals are buried in forum posts. Treasury data lives in blockchain explorers. Security alerts come from Twitter."
> *(pause)*
> "What if your DAO wasn't a dashboard... but a universe?"

[Click: App loads. Cinematic intro: "DAO COSMOS OS" title fades in. Particles converge. Camera flies into the neural network.]

Act 1 [0:20 - 0:50] — THE NEURAL NETWORK

> "This is DAO Cosmos OS. Every node you see is a real entity in the Uniswap DAO — governance
>
> proposals, treasury wallets, contributors, smart contracts. The connections between them represent real on-chain relationships."

[Hover over nodes. Show labels appearing. Show a pulse traveling through connections.]

> "When something happens on-chain — a vote, a transfer, a new delegate — you see it in real-time."

[Demo mode fires a vote event. Show the pulse animation.]

Act 2 [0:50 - 1:20] — GOVERNANCE ARENA

> "Let's zoom into the Governance Arena."

[Click Governance in sidebar. Camera flies to governance cluster. Holographic proposal cards appear.]

> "Every proposal becomes a battle card. You can see vote distribution, whale detection, and AI-generated impact scores — all in real time."

[Click on a proposal. Show expanded view with vote bar, whale indicators, AI summary.]

> "Notice the whale indicator — when a wallet with over 1 million UNI votes, the system flags it immediately."

Act 3 [1:20 - 1:50] — AI COMMANDER

> "But the real power is the AI Commander."

[Click CommandBar. Type: "What is the biggest risk to this DAO right now?"]

[AI responds with streaming text. Relevant nodes glow in the 3D scene.]

> "The AI analyzes real on-chain data and highlights risks directly in the universe. No more reading reports — you see the problem."

Act 4 [1:50 - 2:20] — TREASURY REACTOR

> "The Treasury Reactor visualizes $2.3 billion in DAO assets as a fusion reactor."

[Camera flies to treasury. Reactor core pulses. Energy streams flow.]

> "Inflows and outflows appear as energy streams. If risk increases — the reactor overheats."

[Show risk gauge. Briefly show flow diagram.]

Act 5 [2:20 - 2:50] — MULTIVERSE SIMULATOR

> "And here's where it gets wild. The Multiverse Simulator."

[Navigate to Simulator. Select "Market Crash 50%". Click Simulate.]

> "Using AI, we simulate three possible futures for the DAO."

[Show timeline fork animation. Three timelines diverge. Show result panels.]

> "Timeline A: the DAO adapts and survives. Timeline C: complete collapse. Now DAO operators can prepare instead of react."

Closing [2:50 - 3:00] — THE CLOSER

> "DAO Cosmos OS. Stop reading your DAO. Start experiencing it."

[Camera pulls back to show full universe. Music swell if applicable.]

25.2 Judge Q&A Preparation

Anticipated questions and prepared answers:

Q: "Is this real blockchain data?" A: "Yes. We're pulling live data from the Uniswap DAO governance contracts and treasury via Alchemy SDK. The proposals, vote counts, and treasury balances are all on-chain."

Q: "How does the AI agent work?" A: "We use Claude's API with role-specific system prompts. Each agent role (governance, treasury, security) gets a prompt loaded with current DAO data as context. The AI analyzes real data, not generic information."

Q: "Can this scale to other DAOs?" A: "Absolutely. The architecture is modular. You provide a governor contract address and treasury address, and the system adapts. The 3D visualization, AI agents, and security modules are all DAO-agnostic."

Q: "What's the technical stack?" A: "Next.js 14, React Three Fiber for 3D, Framer Motion for 2D animation, Zustand for state management, Claude API for AI, Alchemy SDK for blockchain data, and Supabase for the database layer."

Q: "What would you build next?" A: "Multi-DAO view — imagine seeing Uniswap, Aave, and Compound as neighboring galaxies in one universe. Also, governance replay with historical timeline

Compound as neighboring galaxies in one universe. Also, governance replay with historical timeline scrubbing, and expanding the gamification system into a full DAO contributor reputation protocol."

---

26. RISK MITIGATION & CONTINGENCY PLANS

Risk	Probability	Impact	Mitigation
Alchemy API down	Low	High	Mock data fallback always loaded. Switch to mock if API fails.
Claude API slow	Medium	Medium	Pre-cache common AI responses. Show "Analyzing..." animation during wait.
Three.js performance crash	Medium	Critical	Cap nodes at 200, particles at 3000. Test on low-end hardware early.
Supabase connection fails	Low	Medium	All data can work from mock. Supabase is nice-to-have, not critical.
Merge conflict disaster	Medium	High	Person A and Person B work on completely separate files. Merge to main only when working.
Demo laptop crashes	Low	Critical	Have backup laptop. Have backup video recording. Deploy to Vercel so any laptop with a browser works.
Judge's browser doesn't support WebGL	Low	Critical	Add WebGL detection. Show video fallback.
Run out of time	High	High	Follow the priority kill list from Section 1. Cut features, not quality.
Both devs exhausted	High	Medium	Enforce 6+ hour sleep. No all-nighter. Tired devs write bugs.

---

27. POST-HACKATHON ROADMAP

If this project wins or gets traction:

Month 1: Multi-DAO support. Add Aave, Compound, ENS DAOs. Allow users to switch between DAO universes.

Month 2: Mobile companion app. Simplified 2D view with push notifications for governance events and security alerts.

Month 3: Full gamification protocol. On-chain reputation system. Soulbound tokens for badges and achievements.

Month 4: DAO-to-DAO visualization. See cross-DAO relationships. Treasury flows between DAOs. Shared contributors appearing as stars in multiple galaxies.

Month 5: Governance replay system. Full historical timeline with scrubbing. Watch how any past proposal played out in real-time.

Month 6: Public launch. Open source the core visualization engine. Plugin system for community-built modules.

---

28. APPENDIX: KEY CODE SNIPPETS

28.1 Tailwind Config

```typescript
// tailwind.config.ts
import type { Config } from 'tailwindcss';

const config: Config = {
  content: ['./src/**/*.{js,ts,jsx,tsx,mdx}'],
  theme: {
    extend: {
      colors: {
        cosmos: {
          void: '#0a0a0f',
          'void-light': '#12121a',
          'void-lighter': '#1a1a2e',
          cyan: '#00f0ff',
          'cyan-dim': '#00a5b0',
          magenta: '#ff2d6a',
          'magenta-dim': '#b01e4a',
          gold: '#ffd700',
          'gold-dim': '#b09500',
          purple: '#7b2ff7',
          'purple-dim': '#5a1fb5',
          green: '#00ff88',
          'green-dim': '#00b060',
        },
      },
      fontFamily: {
        display: ['Orbitron', 'monospace'],
        body: ['Exo 2', 'sans-serif'],
        mono: ['Space Mono', 'monospace'],
      },
      animation: {
        'pulse-slow': 'pulse 3s ease-in-out infinite',
        'float': 'float 4s ease-in-out infinite',
```

```
      'scan': 'scan 3s linear infinite',
      'flicker': 'flicker 4s linear infinite',
    },
    keyframes: {
      float: {
        '0%, 100%': { transform: 'translateY(0px)' },
        '50%': { transform: 'translateY(-6px)' },
      },
      scan: {
        '0%': { transform: 'translateY(-100%)' },
        '100%': { transform: 'translateY(100%)' },
      },
      flicker: {
        '0%, 97%, 100%': { opacity: '1' },
        '98%': { opacity: '0.8' },
        '99%': { opacity: '0.9' },
      },
    },
    backdropBlur: {
      xs: '2px',

    },
  },
},
plugins: [],
};

export default config;
```

28.2 Global CSS (Key Parts)

```
/* globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

@font-face {
  font-family: 'Orbitron';
  src: url('/fonts/Orbitron-Regular.woff2') format('woff2');
  font-weight: 400;
  font-display: swap;
}
@font-face {
  font-family: 'Orbitron';
  src: url('/fonts/Orbitron-Bold.woff2') format('woff2');
```

```
  font-weight: 700;
  font-display: swap;
}
@font-face {
  font-family: 'Exo 2';
  src: url('/fonts/Exo2-Regular.woff2') format('woff2');
  font-weight: 400;
  font-display: swap;
}
@font-face {
  font-family: 'Space Mono';
  src: url('/fonts/SpaceMono-Regular.woff2') format('woff2');
  font-weight: 400;
  font-display: swap;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: #0a0a0f;
  color: #e0e8ff;
  font-family: 'Exo 2', sans-serif;
  overflow: hidden; /* No page scrolling — all content is within panels */
}

/* Holographic panel base class */
.holo-panel {
  background: rgba(10, 10, 20, 0.75);
  backdrop-filter: blur(12px) saturate(150%);
  border: 1px solid rgba(0, 240, 255, 0.15);
  position: relative;
  overflow: hidden;
}

.holo-panel::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: repeating-linear-gradient(
    0deg,
    transparent,
    transparent 2px,
```